

**Informe:**  
**GYM OLYMPO**

**Alumnos: Joscelynné Díaz Z.**  
**Joaquín Medina V.**  
**Abraham González Sa.**  
**Marcelo Mancilla V.**

**Asignatura: Taller de programación - 300D**  
**CRISTIAN ANDRES MORENO SANHUEZA**

**Santiago, junio de 2026.**

# ÍNDICE

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Contexto del proyecto</b>                               | <b>4</b>  |
| <b>Descripción del problema u oportunidad</b>              | <b>4</b>  |
| Problemática                                               | 4         |
| Causas                                                     | 4         |
| Efectos                                                    | 4         |
| Oportunidad                                                | 5         |
| Objetivo general                                           | 5         |
| Objetivos Específicos                                      | 5         |
| Tabla de requerimientos                                    | 5         |
| Tabla de homologación                                      | 8         |
| <b>Situación inicial</b>                                   | <b>8</b>  |
| <b>Alcance del sistema</b>                                 | <b>9</b>  |
| <b>Entregables</b>                                         | <b>9</b>  |
| <b>Restricciones y Supuestos</b>                           | <b>9</b>  |
| <b>Planificación del proyecto</b>                          | <b>9</b>  |
| Fases del proyecto                                         | 10        |
| Inicio                                                     | 10        |
| Desarrollo                                                 | 10        |
| Cierre                                                     | 10        |
| <b>Arquitectura propuesta</b>                              | <b>10</b> |
| Diagrama de arquitectura                                   | 11        |
| <b>Diseño de la Solución</b>                               | <b>12</b> |
| Diagrama de casos de uso                                   | 12        |
| Diagrama de clases                                         | 13        |
| Diagrama de entidad-relación                               | 16        |
| Diagrama de componentes                                    | 19        |
| Diagrama de navegación                                     | 21        |
| Diagrama de despliegue                                     | 23        |
| <b>Configuración de Entornos, Servidores y Continuidad</b> | <b>24</b> |
| <b>Tecnologías utilizadas y justificación</b>              | <b>24</b> |
| Frontend                                                   | 24        |
| Backend                                                    | 25        |
| Base de datos / Backend as a Service                       | 26        |
| <b>Herramientas</b>                                        | <b>26</b> |
| <b>Tabla de Stack Tecnológico</b>                          | <b>26</b> |
| <b>Viabilidad Técnica del Stack Tecnológico</b>            | <b>27</b> |
| <b>Visión de la Situación Futura</b>                       | <b>28</b> |
| <b>Estrategia de Respaldo y Recuperación</b>               | <b>28</b> |
| Configuración del Ambiente de Pruebas                      | 28        |
| Respaldo y Validación de Datos                             | 29        |
| <b>Servicios cloud y justificación</b>                     | <b>29</b> |
| Modelo BaaS (Backend as a Service)                         | 29        |
| Modelo SaaS (Software as a Service)                        | 30        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| Beneficios del uso de Cloud Computing                   | 30        |
| <b>Conceptualización de la solución</b>                 | <b>30</b> |
| Factibilidad de cumplimiento                            | 30        |
| <b>Propósito de la solución y atributos de calidad</b>  | <b>31</b> |
| Propósito de la solución                                | 31        |
| Valor agregado                                          | 31        |
| Atributos de calidad                                    | 31        |
| <b>Plan de Calidad</b>                                  | <b>31</b> |
| Definiciones de Calidad del Proyecto                    | 31        |
| Casos de pruebas                                        | 32        |
| <b>ESTRATEGIA DE EJECUCIÓN DE PRUEBAS</b>               | <b>46</b> |
| Resultados de QA                                        | 46        |
| <b>Definición de metodología</b>                        | <b>49</b> |
| Metodología de desarrollo (Scrum)                       | 49        |
| Roles del Proyecto                                      | 50        |
| Patrón arquitectónico basado en Componentes y Servicios | 51        |
| <b>Documentación y entregable</b>                       | <b>51</b> |
| Evidencia de Avance del Proyecto                        | 52        |
| Documentación de Pruebas Realizadas                     | 53        |
| Stakeholders Identificados                              | 54        |
| <b>Conclusión</b>                                       | <b>54</b> |
| Bibliografía                                            | 54        |

# Contexto del proyecto

El proyecto corresponde al desarrollo de una solución informática para el cliente Gym Olympo, un gimnasio con aproximadamente dos años de funcionamiento, ubicado en un sector rural. Este establecimiento opera bajo un modelo de atención con cupos limitados por horario, ofreciendo a sus clientes planes de sesiones semanales y mensuales, los cuales deben ser utilizados dentro de un período determinado.

Actualmente, la gestión de las sesiones, horarios y disponibilidad se realiza de manera manual, lo que dificulta la visualización de cupos disponibles en tiempo real y genera problemas en la organización de la agenda. Esta situación afecta tanto al administrador como a los clientes, provocando desorden en la planificación de las sesiones y posibles conflictos por sobrecupo.

En este contexto, se identifica la necesidad de implementar una plataforma digital que permita optimizar la gestión interna del gimnasio, facilitando el control de horarios, la administración de clientes y la organización de sesiones, mejorando así la eficiencia operativa y la experiencia de los usuarios.

## Descripción del problema u oportunidad

### Problemática

El gimnasio Gym Olympo actualmente gestiona sus reservas, sesiones y asignación de horarios de forma manual, lo que genera una alta carga operativa para el personal y una mayor probabilidad de errores en la administración de los cupos.

Debido a la falta de un sistema automatizado, el control de las sesiones dentro del período contratado no se realiza de manera eficiente. Esto provoca que algunos clientes intenten recuperar sesiones fuera del plazo establecido, generando sobrecupo en ciertos horarios y desorganización en la planificación general.

Asimismo, el proceso de agendamiento no está digitalizado, lo que obliga a los clientes a realizar reservas de forma presencial o mediante contacto directo, limitando la rapidez, accesibilidad y disponibilidad del servicio.

### Causas

- Falta de un sistema digital para la gestión de sesiones y reservas.
- Dependencia de procesos manuales para el control de agendas.
- Ausencia de automatización en el control de cupos y sesiones.
- Limitado acceso a canales digitales de agendamiento.

### Efectos

- Sobrecarga administrativa para el personal.

- Mayor probabilidad de errores en la asignación de horarios.
- Pérdida de control sobre las sesiones contratadas.
- Sobrecupo en determinados horarios.
- Mala experiencia para los clientes.
- Pérdida de clientes potenciales.
- Limitación en el crecimiento del gimnasio.

## Oportunidad

La problemática identificada representa una oportunidad para desarrollar una plataforma digital que permita automatizar la gestión de sesiones, reservas y control de cupos del gimnasio.

Esta solución permitirá a los clientes agendar sesiones de manera online, visualizar disponibilidad en tiempo real y gestionar sus reservas de forma autónoma, mejorando significativamente la experiencia de usuario.

Por otro lado, facilitará la administración interna al reducir la carga operativa, minimizar errores y optimizar el uso de los recursos, permitiendo además futuras integraciones y mejoras en el sistema.

## Objetivo general

Desarrollar una aplicación web basada en arquitectura cloud que permita digitalizar la gestión de reservas del gimnasio, automatizar la administración de la agenda y optimizar la organización de horarios y el control de asistencia de los usuarios.

## Objetivos Específicos

- Diseñar e implementar un sistema de agendamiento en línea que permita a los usuarios reservar sesiones y asesorías según la disponibilidad de horarios.
- Automatizar la gestión de la agenda del gimnasio, controlando de manera eficiente los cupos por horario y evitando el sobrecupo, reduciendo la intervención manual y minimizando errores en las reservas.
- Implementar un sistema de validación de reservas mediante confirmación de pago por transferencia.
- Implementar reglas de negocio que aseguren el uso de sesiones únicamente dentro del período contratado, evitando recuperaciones fuera de plazo.
- Desarrollar un módulo de administración que permita gestionar clientes, planes, horarios, reservas y disponibilidad.
- Integrar un sistema de registro y validación de pagos, asociado a la confirmación de reservas.
- Mejorar la experiencia del usuario mediante una interfaz intuitiva, accesible y disponible 24/7 desde dispositivos móviles o web.

## Tabla de requerimientos

Funcionales:

| ID    | Tipo      | Categoría     | Requerimiento              | Descripción                                                                 | Prioridad | Estado        |
|-------|-----------|---------------|----------------------------|-----------------------------------------------------------------------------|-----------|---------------|
| RF-01 | Funcional | Cliente       | Ver sesiones               | El cliente puede visualizar sus sesiones disponibles dentro del plan        | Alta      | Pendiente     |
| RF-02 | Funcional | Cliente       | Agendar sesiones           | El cliente puede reservar sesiones según disponibilidad de cupos            | Alta      | Pendiente     |
| RF-03 | Funcional | Cliente       | Agendar asesorías          | El cliente puede reservar asesorías cada 15 días                            | Alta      | Pendiente     |
| RF-04 | Funcional | Administrador | Registrar clientes         | El administrador puede crear y gestionar clientes                           | Alta      | En desarrollo |
| RF-05 | Funcional | Administrador | Gestionar agenda           | El administrador puede ver y modificar la agenda de sesiones                | Alta      | En desarrollo |
| RF-06 | Funcional | Sistema       | Control de cupos           | El sistema limita automáticamente los cupos por horario                     | Alta      | En desarrollo |
| RF-07 | Funcional | Sistema       | Control de sesiones        | Validar que las sesiones se usen dentro del período contratado              | Alta      | En desarrollo |
| RF-08 | Funcional | Sistema       | Bloqueo de recuperaciones  | No permitir recuperar sesiones fuera del período                            | Alta      | En desarrollo |
| RF-09 | Funcional | Sistema       | Registro de pagos          | Registrar pagos asociados a planes                                          | Media     | Pendiente     |
| RF-10 | Funcional | Sistema       | Gestión de planes          | Administrar planes de 12 sesiones en 4 semanas                              | Alta      | Pendiente     |
| RF-11 | Funcional | Autenticación | Registro de usuario        | El usuario puede crear una cuenta en el sistema mediante email y contraseña | Alta      | En desarrollo |
| RF-12 | Funcional | Autenticación | Inicio de sesión           | El usuario puede iniciar sesión y acceder a su cuenta                       | Alta      | En desarrollo |
| RF-13 | Funcional | Autenticación | Recuperación de contraseña | El usuario puede recuperar su contraseña mediante email                     | Media     | En desarrollo |
| RF-14 | Funcional | Cliente       | Visualizar disponibilidad  | El cliente puede ver horarios disponibles en tiempo real                    | Alta      | Pendiente     |
| RF-15 | Funcional | Cliente       | Cancelar reservas          | El cliente puede cancelar sesiones dentro del período permitido             | Alta      | Pendiente     |
| RF-16 | Funcional | Sistema       | Validación de pago         | El sistema valida reservas solo si el pago ha sido confirmado               | Alta      | Pendiente     |
| RF-17 | Funcional | Sistema       | Notificación de reserva    | El sistema notifica al usuario al confirmar una reserva                     | Media     | Pendiente     |
| RF-18 | Funcional | Sistema       | Control de asistencia      | Registrar asistencia del cliente a sesiones agendadas                       | Alta      | Pendiente     |
| RF-19 | Funcional | Administrador | Editar planes              | El administrador puede modificar planes existentes                          | Media     | En desarrollo |
| RF-20 | Funcional | Administrador | Eliminar                   | El administrador puede eliminar reservas                                    | Media     | En            |

|       |           |               |                                    |                                                                            |       |               |
|-------|-----------|---------------|------------------------------------|----------------------------------------------------------------------------|-------|---------------|
|       |           | dor           | reservas                           | en casos excepcionales                                                     |       | desarrollo    |
| RF-21 | Funcional | Administrador | Reportes                           | Generar reportes de uso de sesiones y ocupación                            | Media | En desarrollo |
| RF-22 | Funcional | Sistema       | Validación de cupos en tiempo real | El sistema actualiza cupos automáticamente al reservar                     | Alta  | Pendiente     |
| RF-23 | Funcional | Sistema       | Historial del cliente              | El cliente puede ver historial de sesiones y pagos                         | Media | Pendiente     |
| RF-24 | Funcional | Sistema       | Gestión de roles                   | Diferenciar permisos entre cliente y administrador                         | Alta  | Pendiente     |
| RF-25 | Funcional | Sistema       | Servicios HTTPS                    | expuestos mediante Firebase security rules para la gestión segura de datos | Alta  | Pendiente     |

No funcionales:

| ID     | Tipo         | Categoría      | Requerimiento       | Descripción                                                                                                                  | Prioridad | Estado        |
|--------|--------------|----------------|---------------------|------------------------------------------------------------------------------------------------------------------------------|-----------|---------------|
| RNF-01 | No Funcional | Seguridad      | Seguridad alta      | Protección de datos personales y accesos                                                                                     | Alta      | Pendiente     |
| RNF-02 | No Funcional | Usabilidad     | Interfaz simple     | Sistema fácil de usar para adultos y adultos mayores                                                                         | Alta      | En desarrollo |
| RNF-03 | No Funcional | Accesibilidad  | Acceso móvil        | Compatible con dispositivos móviles                                                                                          | Alta      | En desarrollo |
| RNF-04 | No Funcional | Rendimiento    | Tiempo de respuesta | El sistema debe responder en menos de 3 segundos                                                                             | Alta      | En desarrollo |
| RNF-05 | No Funcional | Disponibilidad | Alta disponibilidad | El sistema debe estar disponible 24/7                                                                                        | Alta      | En desarrollo |
| RNF-06 | No Funcional | Escalabilidad  | Escalabilidad cloud | El sistema debe soportar aumento de usuarios sin fallos                                                                      | Alta      | En desarrollo |
| RNF-07 | No Funcional | Seguridad      | Autenticación JWT   | Uso de tokens seguros para acceso al sistema                                                                                 | Alta      | En desarrollo |
| RNF-08 | No Funcional | Seguridad      | Protección de datos | Los datos no deben ser accesibles desde el frontend directamente                                                             | Alta      | En desarrollo |
| RNF-09 | No Funcional | Compatibilidad | Multiplataforma     | Funciona en navegador y dispositivos móviles                                                                                 | Alta      | En desarrollo |
| RNF-10 | No Funcional | Mantenibilidad | Serverless          | El sistema debe mantener una estructura modular basada en componentes Angular, facilitando el mantenimiento y escalabilidad. | Media     | Pendiente     |
| RNF-11 | No Funcional | Usabilidad     | Facilidad de uso    | Usuarios pueden aprender a usar el sistema en menos de 10 min                                                                | Alta      | Pendiente     |

|        |              |                 |                     |                                                          |       |           |
|--------|--------------|-----------------|---------------------|----------------------------------------------------------|-------|-----------|
| RNF-12 | No Funcional | Confiabilidad   | Tolerancia a fallos | El sistema debe manejar errores sin caerse               | Alta  | Pendiente |
| RNF-13 | No Funcional | Infraestructura | Cloud               | El sistema debe estar desplegado en servicios cloud      | Alta  | Pendiente |
| RNF-14 | No Funcional | Seguridad       | Control de acceso   | Validación de permisos según rol                         | Alta  | Pendiente |
| RNF-15 | No Funcional | Backup          | Respaldo de datos   | El sistema debe tener respaldo automático de información | Media | Pendiente |

## Tabla de homologación

| Sistema / Competencia                   | ¿Qué ofrece?                                                            | Problemas / Limitaciones                                                           | Diferencia con nuestra solución                                                        |
|-----------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Agenda manual (actual)                  | Gestión de reservas de forma presencial o por contacto directo          | Alta probabilidad de errores, sobrecupo, desorganización, dependencia del personal | Automatización completa del sistema, control en tiempo real y reducción de errores     |
| Google Calendar / Agenda digital básica | Permite agendar horarios y visualizar disponibilidad                    | No controla cupos automáticamente ni gestiona sesiones por planes                  | Sistema especializado con control de cupos, validación de sesiones y reglas de negocio |
| Mindbody (software fitness)             | Plataforma completa de gestión de gimnasios (pagos, reservas, clientes) | Alto costo, complejidad para pymes, poco adaptable a necesidades específicas       | Solución personalizada, adaptable y de menor costo para el contexto del cliente        |
| Reservas por WhatsApp                   | Comunicación directa con clientes para coordinar horarios               | Desorden en la información, difícil seguimiento, no hay automatización             | Plataforma centralizada con historial, control y acceso en tiempo real                 |
| Aplicaciones genéricas de reservas      | Permiten reservar servicios online                                      | No consideran reglas específicas como sesiones por periodo o control de planes     | Sistema diseñado específicamente para el modelo de negocio del gimnasio                |

La tabla de homologación permite establecer la relación entre los requerimientos definidos y los componentes de la solución tecnológica propuesta, asegurando trazabilidad entre las necesidades del sistema y su implementación.

## Situación inicial

Actualmente, el gimnasio Gym Olympo gestiona su agenda de manera manual de forma presencial y utilizando métodos tradicionales para la asignación de horarios. Este proceso al ser presencial genera una alta dependencia del personal para organizar las reservas y atender a los usuarios, aumentando la probabilidad de errores en la asignación de horarios y en el control de las sesiones.

Asimismo, la gestión manual dificulta el seguimiento del uso de las sesiones dentro del período contratado, lo que provoca que algunos clientes intenten reagendar fuera de plazo, generando sobrecupo y desorganización en la planificación de la agenda.

# Alcance del sistema

El proyecto contempla el desarrollo de una plataforma digital orientada a la gestión de sesiones, reservas y asesorías del gimnasio, en coherencia con los objetivos planteados.

Incluye:

- Sistema de registro e inicio de sesión de usuarios.
- Gestión de perfiles de clientes.
- Sistema de agendamiento de sesiones en línea.
- Control automático de cupos por horario.
- Gestión de planes de entrenamiento y sesiones disponibles.
- Agenda de asesorías exclusivamente en línea.
- Validación de reservas mediante confirmación de pago.
- Módulo de administración para gestión de clientes, horarios y reservas.
- Base de datos en la nube.

No incluye:

- Integración con dispositivos físicos.
- Control biométrico de asistencia.
- Automatización de equipamiento del gimnasio.

## Entregables

- Aplicación web funcional de gestión de reservas.
- Backend serverless implementado mediante Firebase
- Base de datos configurada en la nube.
- Documentación técnica del sistema.
- Prototipo de interfaz de usuario (mockup).

## Restricciones y Supuestos

El desarrollo de la aplicación web se encuentra sujeto a restricciones de tiempo y a recursos limitados, considerando que el proyecto se implementa en el contexto de una pyme. Asimismo, el correcto funcionamiento del sistema dependerá de la disponibilidad de conexión a internet.

Se asume que los usuarios cuentan con acceso a dispositivos como teléfonos móviles o notebooks, así como con conexión a internet, lo que permitirá realizar reservas y utilizar las funcionalidades de la plataforma de manera adecuada.

## Planificación del proyecto

El desarrollo del proyecto se realizará bajo el marco de trabajo Scrum, permitiendo una gestión ágil, iterativa e incremental, facilitando la adaptabilidad a cambios.

El proyecto se dividirá en sprints de dos semanas, donde en cada iteración se planifican, desarrollan, prueban y revisan funcionalidades específicas del sistema. Esto permite obtener retroalimentación

continúa y realizar ajustes oportunos, además, en cada sprint se realizan reuniones de planificación, seguimiento y revisión, con el fin de asegurar el cumplimiento de los objetivos definidos.

## Fases del proyecto

### Inicio

Realizaremos una reunión inicial con el cliente para poder realizar los requerimientos correspondientes, en base a esto, se define una arquitectura serverless basada en Firebase.

### Desarrollo

En esta etapa se profundiza el análisis de requerimientos con el cliente con el objetivo de levantar y analizar los requerimientos del sistema. A partir de esta información, se define una arquitectura serverless basada en Firebase, Firebase Authentication gestiona la identidad de los usuarios y Cloud Firestore almacena la información del sistema, donde el backend centraliza la lógica de negocio a través de módulos internos organizados por responsabilidad y Firebase actúa como capa de infraestructura para identidad y persistencia de datos, permitiendo una organización modular basada en responsabilidades internas del sistema.

- Desarrollo del frontend (interfaz de usuario) utilizando Ionic..
- Integración entre el frontend y el backend.
- Implementación de la base de datos en la nube.

Desarrollo de funcionalidades clave, tales como:

- Sistemas de reservas.
- Control de cupos.
- Gestión de usuarios.
- Módulo de administración.

Y ejecución de pruebas continuas en cada iteración o sprint, con el fin de asegurar la calidad del sistema.

### Cierre

En esta fase se llevará a cabo la validación final del sistema previo a su entrega. Esto incluye:

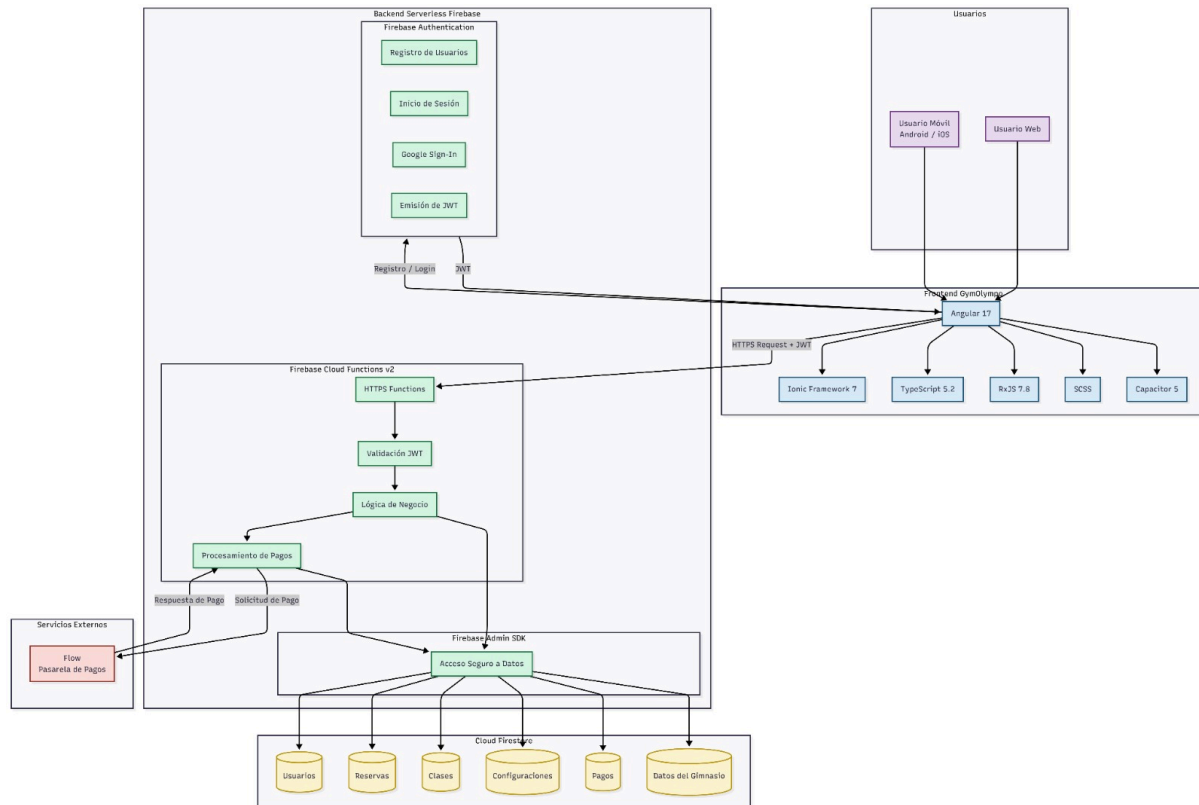
- Pruebas finales del sistema.
- Corrección de errores detectados.
- Validación de funcionalidades en base a los requerimientos establecidos.
- Elaboración de documentación técnica.
- Entrega de la aplicación web.

## Arquitectura propuesta

La arquitectura propuesta corresponde a una arquitectura Serverless basada en Firebase, donde Firebase Authentication gestiona la identidad de los usuarios y Cloud Firestore almacena la

información del sistema. El frontend desarrollado con Angular e Ionic consume estos servicios mediante funciones HTTPS protegidas por autenticación JWT.

## Diagrama de arquitectura



La arquitectura de GymOlympo está basada en un modelo Serverless utilizando Firebase como plataforma principal de backend. Los usuarios acceden al sistema mediante una aplicación desarrollada con Angular e Ionic, la cual puede ejecutarse tanto en navegadores web como en dispositivos móviles gracias a Capacitor.

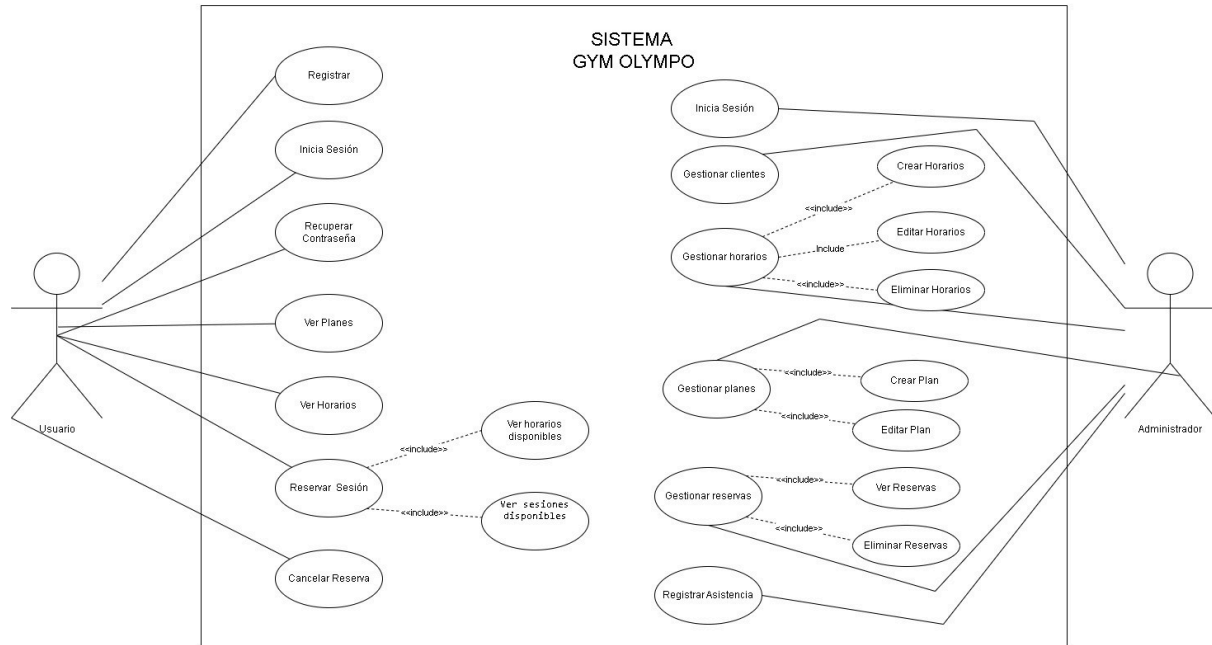
El frontend se comunica directamente con Firebase Authentication para gestionar el registro, inicio de sesión y autenticación mediante Google, obteniendo un token JWT que identifica de forma segura a cada usuario.

Para acceder a la información almacenada, utilizan Firebase Admin SDK, que permite una conexión segura con Cloud Firestore, donde se almacenan los datos de usuarios, reservas, clases, pagos y configuraciones del gimnasio.

Esta arquitectura garantiza una adecuada separación de responsabilidades, mayor seguridad al evitar el acceso directo del cliente a la base de datos y una alta escalabilidad gracias al uso de servicios administrados en la nube.

# Diseño de la Solución

## Diagrama de casos de uso



El diagrama de casos de uso representa los requerimientos funcionales del sistema GymOlympo, ilustrando cómo los diferentes actores interactúan con la plataforma para ejecutar tareas específicas. Permite visualizar el alcance del sistema y las dependencias entre las distintas funcionalidades.

### ACTORES DEL SISTEMA:

El sistema define dos actores principales que interactúan con la frontera del aplicativo:

**Usuario:** Representa al cliente, alumno o atleta del gimnasio que consume los servicios básicos de la plataforma.

**Administrador:** Representa al personal de gestión y soporte encargado del control operativo, comercial y de supervisar las actividades generales del gimnasio.

### CASOS DE USO DEL ACTOR: USUARIO:

Este actor interactúa con las funciones orientadas al autoservicio y la gestión de sus propias actividades dentro del gimnasio:

- **Registrar:** Permite a un nuevo usuario darse de alta en la plataforma.
- **Inicia Sesión:** Permite el acceso seguro al sistema mediante credenciales válidas.
- **Recuperar Contraseña:** Proceso de soporte en caso de olvido de credenciales de acceso.
- **Ver Planes:** Consulta de la oferta comercial y membresías del gimnasio.
- **Ver Horarios:** Visualización de la agenda o calendario general de clases.

- Reservar Sesión: Funcionalidad clave para agendar una clase. Este caso de uso requiere de manera obligatoria la ejecución de otros subprocesos mediante relaciones de inclusión (include):
  - Incluye: Ver horarios disponibles (para validar huecos en la agenda).
  - Incluye: Ver sesiones disponibles (para corroborar el aforo o cupos del bloque).
- Cancelar Reserva: Permite al usuario liberar su cupo en una clase previamente agendada.

## CASOS DE USO DEL ACTOR: ADMINISTRADOR:

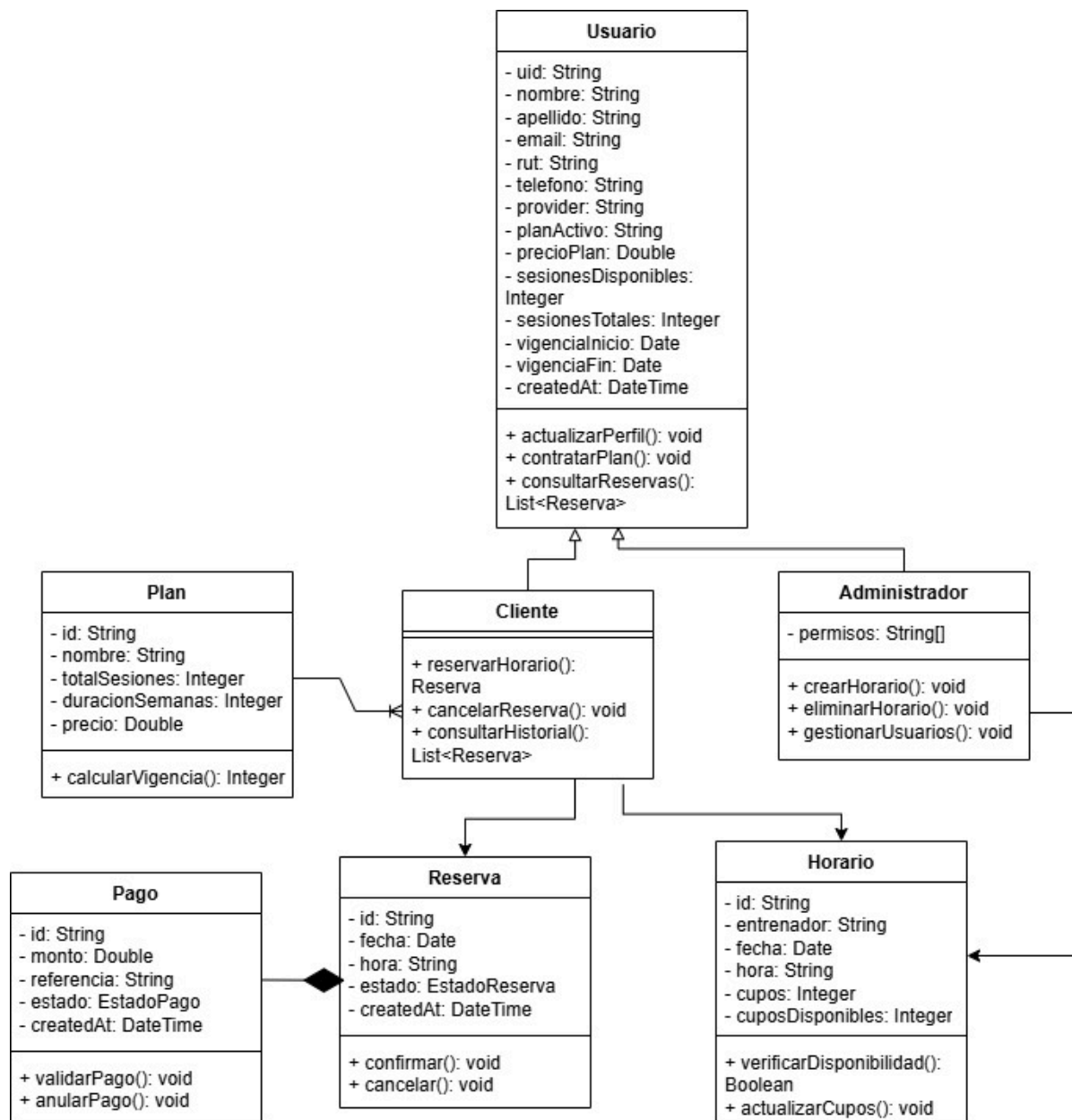
Este actor posee facultades globales para gestionar los módulos críticos de la plataforma:

- Inicia Sesión: Acceso autenticado al panel de control administrativo.
- Gestionar clientes: Permite visualizar, dar de alta, modificar o de baja los perfiles de los usuarios del sistema.
- Registrar Asistencia: Control en tiempo real de la asistencia presencial de los alumnos a las clases.
- Gestionar horarios: Modulo central para la administración de la agenda, el cual incluye obligatoriamente (include) las operaciones básicas:
  - Incluye: Crear Horarios
  - Incluye: Editar Horarios
  - Incluye: Eliminar Horarios
- Gestionar planes: Control de la oferta comercial de membresías, incluyendo las operaciones obligatorias (include):
  - Incluye: Crear Plan
  - Incluye: Editar Plan
- Gestionar reservas: Permite supervisar las citas agendadas por los clientes, incluyendo las operaciones obligatorias (include):
  - Incluye: Ver Reservas
  - Incluye: Eliminar Reservas

## SIGNIFICADO DE LAS RELACIONES DE INCLUSIÓN (INCLUDE)

Las líneas discontinuas con la etiqueta <> indican que el caso de uso base no puede completarse de manera exitosa sin ejecutar obligatoriamente el caso de uso apuntado. Por ejemplo, un Administrador no puede "Gestionar planes" de manera abstracta, sino que dicha gestión se materializa obligatoriamente a través de las acciones de "Crear Plan" o "Editar Plan". De igual forma, el Usuario no puede "Reservar Sesión" sin que el sistema realice primero la consulta interna de "Ver horarios disponibles" y "Ver sesiones disponibles".

## Diagrama de clases



El diagrama de clases presenta la arquitectura lógica y estructural del sistema de gestión para el gimnasio GymOlympo. En él se definen las entidades principales, sus atributos, los métodos que determinan su comportamiento y las relaciones que permiten el flujo de la información dentro de la plataforma.

## DESCRIPCIÓN DE LAS CLASES Y MÓDULOS PRINCIPALES

Clase Usuario (Clase Base) Es la clase transversal y abstracta del sistema que centraliza la información general de cualquier persona registrada en la plataforma.

- Atributos: Almacena datos personales y de autenticación como uid, nombre, apellido, email, rut, telefono y provider. También incluye el control de membresías mediante planActivo, precioPlan, sesionesDisponibles, sesionesTotales, rangos de fechas (vigenciaInicio, vigenciaFin) y la fecha de creación del registro (createdAt).

- Métodos: Define las acciones globales de actualizarPerfil(), contratarPlan() y consultarReservas().

Clase Cliente (Especialización) Representa a los usuarios finales o atletas del gimnasio. Hereda directamente todos los atributos y métodos de la clase Usuario.

- Métodos Propios: Incorpora la lógica de negocio para interactuar con los servicios del gimnasio, tales como reservarHorario(), cancelarReserva() y consultarHistorial().

Clase Administrador (Especialización) Representa al personal encargado de la gestión operativa y de control del gimnasio. Hereda directamente de la clase Usuario.

- Atributos: Cuenta con un arreglo de permisos para segmentar los accesos internos.
- Métodos Propios: Dispone de funciones críticas de control como crearHorario(), eliminarHorario() y gestionarUsuarios().

Clase Plan Define la oferta comercial de membresías que ofrece GymOlympo.

- Atributos: Contiene la identificación del plan (id), nombre, el cupo de clases permitidas (totalSesiones), el tiempo limite (duracionSemanas) y el precio.
- Métodos: Implementa calcularVigencia() para determinar las fechas de vencimiento de las membresías adquiridas de forma automatizada.

Clase Horario Representa la grilla de actividades, clases o turnos disponibles en el gimnasio.

- Atributos: Incluye el identificador (id), el entrenador asignado, la fecha, la hora, los cupos totales de la sala y los cuposDisponibles en tiempo real.
- Métodos: Se encarga de verificarDisponibilidad() antes de procesar una reserva y de actualizarCupos() cada vez que un cliente se inscribe o cancela.

Clase Reserva Es la entidad que materializa la intención de un cliente de asistir a una clase en un horario determinado.

- Atributos: Registra un id, la fecha y hora de la cita, el estado (utilizando el enumerador EstadoReserva) y la fecha de creación del registro (createdAt).
- Métodos: Permite las operaciones de confirmar() y cancelar() la asistencia.

Clase Pago Módulo encargado del registro financiero de las transacciones dentro de la aplicación.

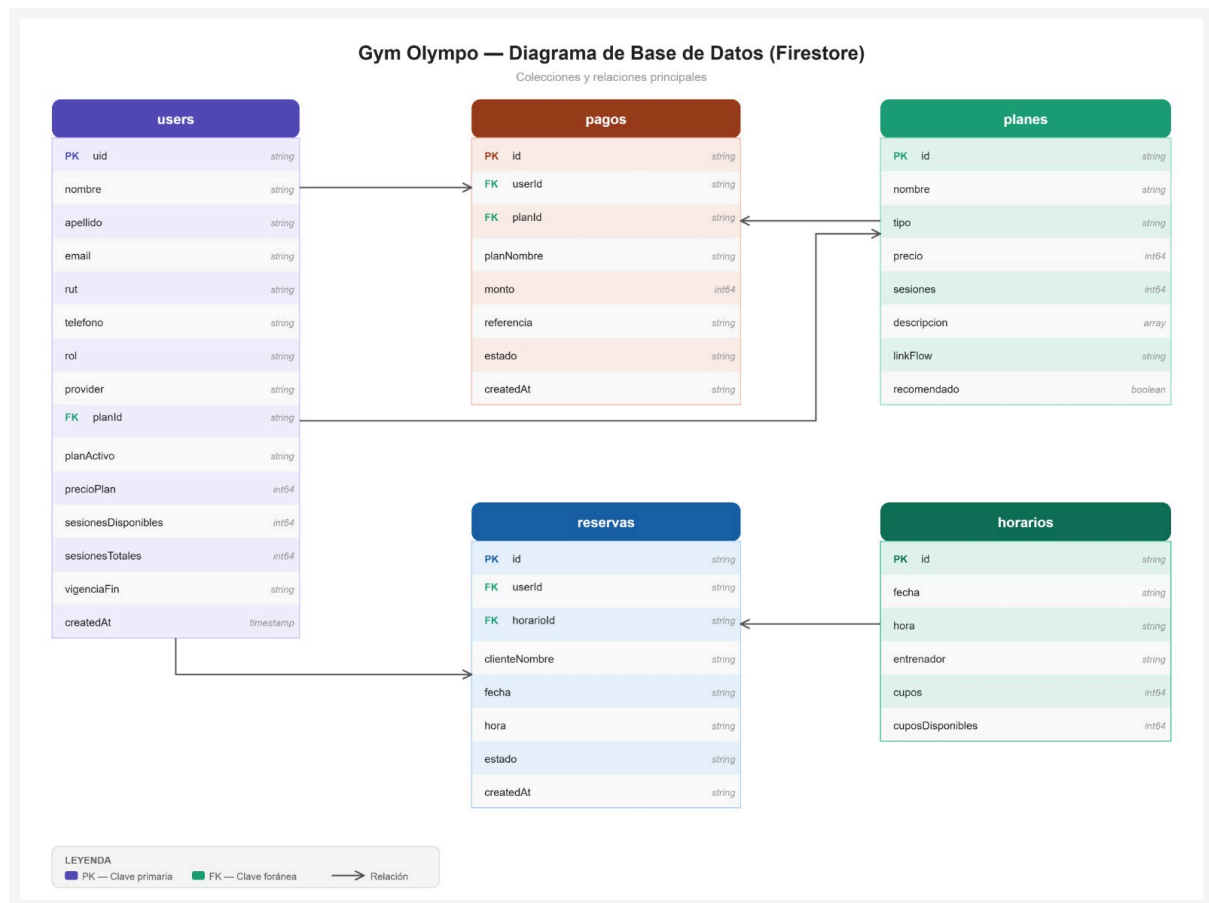
- Atributos: Posee un id, el monto cobrado, la referencia bancaria o de pasarela de pago, el estado (utilizando el enumerador EstadoPago) y el createdAt.
- Métodos: Ejecuta las acciones esenciales de control de cobros mediante validarPago() y anularPago().

## ANÁLISIS DE RELACIONES Y DEPENDENCIAS

- Herencia o Generalización (Flechas con punta de triángulo vacío): Las clases Cliente y Administrador heredan directamente de la clase Usuario. Esto significa que el sistema aplica el principio de reutilización de código, donde ambas clases especializadas adquieren por defecto todas las propiedades de autenticación e identidad de la clase padre.
- Composición (Línea con rombo negro relleno): Existe una relación de composición entre Reserva y Pago (donde el rombo está del lado de Reserva). Esto indica una dependencia de ciclo de vida absoluta: un objeto Pago tiene sentido de existir únicamente si está asociado a una Reserva. Si una reserva se destruye del sistema, el flujo de ciclo de vida del pago asociado se ve afectado directamente.
- Asociación y Navegabilidad (Líneas con flechas directas):
- El Cliente está directamente asociado a un Plan para el consumo de sus sesiones.
- El Cliente se vincula con la Reserva para agendar sus asistencias y consulta la disponibilidad visualizando la clase Horario.
- El Administrador posee una línea de asociación directa hacia Horario, lo que le otorga la facultad de poblar, modificar y controlar la disponibilidad de las clases del gimnasio.

## Diagrama de entidad-relación

Es fundamental destacar que este diagrama tiene un carácter puramente ilustrativo y representativo para la documentación del proyecto. Debido a que GymOlympo utiliza Firebase Firestore como motor de base de datos, la arquitectura real es del tipo NoSQL (no relacional) orientada a colecciones y documentos. Por lo tanto, conceptos tradicionales como Claves Primarias (PK) o Claves Foráneas (FK) se emplean en este modelo únicamente como una abstracción lógica para representar conceptualmente cómo se vinculan las referencias e identificadores entre los distintos documentos.



El diagrama presenta el modelo de datos diseñado para la base de datos NoSQL Firebase Firestore del sistema GymOlympto. A pesar de la naturaleza no relacional de Firestore, el diseño adopta esta estructura de relaciones para asegurar que el flujo y la integridad de la información del gimnasio sean fáciles de entender en el reporte técnico.

#### DESCRIPCIÓN DE LAS COLECCIONES (ENTIDADES):

Colección "users" (Usuarios) Centraliza la información de todas las personas registradas, consolidando en una sola estructura tanto a clientes como a administradores mediante un atributo de rol.

- Identificador de referencia (Representado como PK): uid (Identificador único del usuario provisto por Firebase Authentication).
- Referencia externa (Representada como FK): planId (Enlace al documento del plan de membresía contratado).
- Atributos: nombre, apellido, email, rut, telefono, rol (determina los permisos), provider (método de registro), planActivo, precioPlan, sesionesDisponibles, sesionesTotales, vigenciaFin y createdAt (fecha de creación).

Colección "pagos" (Pagos) Registra el historial financiero y las transacciones de compra de planes dentro de la aplicación.

- Identificador de referencia (Representado como PK): id (Código único del pago).

- Referencias externas (Representadas como FK): userId (Vincula el pago al usuario que lo realizó) y planId (Vincula el pago al plan adquirido).
- Atributos: planNombre, monto, referencia (código de la pasarela de pago Flow), estado y createdAt.

Colección "planes" (Planes) Funciona como el catálogo de membresías y servicios comerciales ofrecidos por el gimnasio.

- Identificador de referencia (Representado como PK): id (Código único del plan).
- Atributos: nombre, tipo, precio, sesiones (total de clases permitidas), descripción (arreglo de textos con los beneficios).

Colección "reservas" (Reservas) Almacena las citas y agendas programadas por los atletas para asistir a sus clases.

- Identificador de referencia (Representado como PK): id (Código único de la reserva).
- Referencias externas (Representadas como FK): userId (Identifica al cliente que reserva) y horarioId (Identifica el bloque horario de la clase).
- Atributos: clienteNombre, fecha, hora, estado (ej. confirmada/cancelada) y createdAt. Nota: Almacena directamente datos como clienteNombre, fecha y hora para optimizar las lecturas en Firestore, aplicando una técnica clásica de desnormalización NoSQL para evitar consultas dobles.

Colección "horarios" (Horarios) Representa la oferta de bloques, turnos o clases disponibles en la agenda diaria del gimnasio.

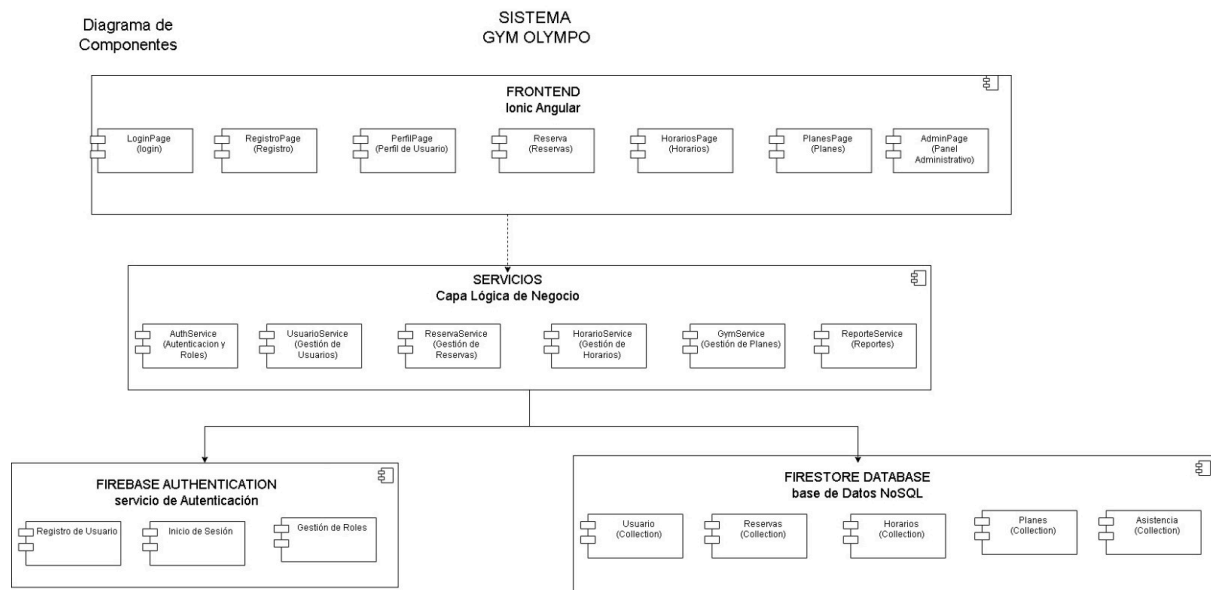
- Identificador de referencia (Representado como PK): id (Código único del bloque).
- Atributos: fecha, hora, entrenador (nombre del profesor), cupos (capacidad total del espacio) y cuposDisponibles (contador en tiempo real).

#### ANÁLISIS DE REFERENCIAS Y FLUJO DE INFORMACIÓN (LÓGICA NOSQL):

A pesar de ser una base de datos NoSQL, el sistema establece conexiones lógicas mediante ID de referencia para estructurar el comportamiento de la aplicación:

- Vínculo de Membresía (users -> planes): Cada documento de la colección "users" posee un campo "planId" que guarda el ID del plan. Esto permite al sistema buscar el documento correspondiente en la colección "planes" y validar los accesos del usuario.
- Vínculo Financiero (pagos -> users / pagos -> planes): La colección de "pagos" actúa como un puente de información guardando el "userId" y el "planId". Esto mapea de forma exacta que cliente generó el ingreso y que producto NoSQL se consumió.
- Vínculo de Agendamiento (reservas -> users / reservas -> horarios): La colección "reservas" une el comportamiento de los clientes con la disponibilidad del gimnasio. Guarda el "userId" para saber quien asistirá y el "horarioId" para saber en qué bloque de la colección "horarios" se restará un cupo en el contador "cuposDisponibles" en tiempo real.

# Diagrama de componentes



El diagrama de componentes detalla la arquitectura de software del sistema GymOlympo organizada en tres capas estructurales bien definidas (Frontend, Servicios de Lógica de Negocio y Backend en la nube). Esta organización permite un desarrollo modular, donde cada componente cumple una función específica y se comunica de manera ordenada con los demás, facilitando el mantenimiento y la escalabilidad de la aplicación.

A continuación, se explica cada una de las capas y sus respectivos componentes:

**CAPA DE INTERFAZ DE USUARIO (FRONTEND - IONIC ANGULAR):** Esta capa superior contiene los componentes visuales con los que interactúan directamente tanto los clientes como los administradores del gimnasio. Está desarrollada bajo el framework Ionic con Angular y se compone de las siguientes pantallas o páginas modulares:

- LoginPage: Componente encargado de gestionar la vista para el inicio de sesión.
- RegistroPage: Gestiona el formulario y la vista de creación de nuevas cuentas de usuario.
- PerfilPage: Permite al usuario visualizar y modificar sus datos personales en pantalla.
- Reserva: Despliega el módulo para que los clientes gestionen y visualicen sus reservas de clases.
- HorariosPage: Muestra el calendario, los bloques y la disponibilidad horaria del gimnasio.
- PlanesPage: Presenta el catálogo comercial de membresías y planes de entrenamiento.
- AdminPage: Panel centralizado exclusivo para el uso del administrador, donde se unifican las herramientas de gestión operativa.

**CAPA DE LÓGICA DE NEGOCIO (SERVICIOS):** Actúa como un mediador o puente intermedio entre la interfaz gráfica (Frontend) y los servicios de la nube (Backend). Su función principal es procesar las peticiones del usuario, aplicar las reglas del negocio del gimnasio y enviar o traer la información desde la base de datos de manera limpia. Contiene los siguientes módulos lógicos:

- AuthService: Maneja las validaciones de acceso, el estado de la sesión del usuario y el control de roles (si es Cliente o Administrador).
- UsuarioService: Ejecuta los procesos relacionados con la consulta y modificación de perfiles o datos de los usuarios.
- ReservaService: Controla la lógica para agendar, verificar cupos y cancelar citas en el gimnasio.
- HorarioService: Procesa la creación, edición y actualización de los bloques y turnos de las clases semanales.
- GymService: Administra las operaciones lógicas ligadas a los planes de suscripción y membresías.
- ReporteService: Consolida e interpreta los datos del sistema para generar las estadísticas e informes requeridos por la administración.

**CAPA DE INFRAESTRUCTURA Y PERSISTENCIA (SERVICIOS BACKEND):** Se ubica en la parte inferior del diagrama y representa los servicios en la nube de Google Firebase que sostienen el funcionamiento lógico e informático del sistema. Se divide en dos grandes subgrupos:

**Módulo Firebase Authentication (Servicio de Autenticación):** Se encarga de la seguridad a nivel de servidor. Procesa los componentes de:

- Registro de Usuario: Validación y creación de credenciales seguras.
- Inicio de sesión: Comprobación de identidad.
- Gestión de Roles: Asignación y validación de permisos de seguridad en el sistema.

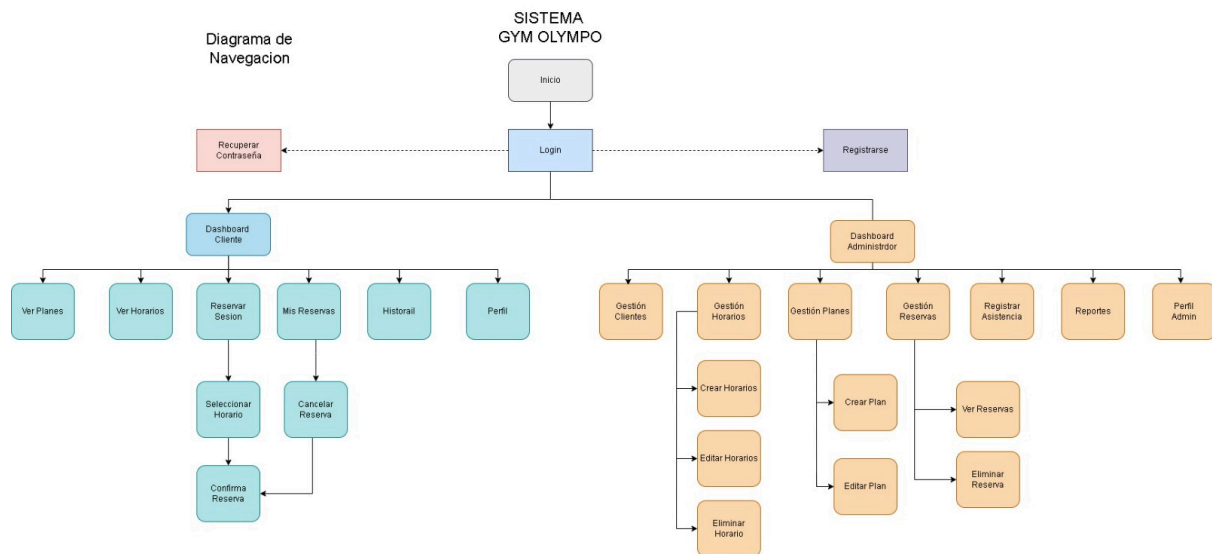
**Módulo Firestore Database (Base de Datos NoSQL):** Es el motor de persistencia donde se almacena toda la información del gimnasio estructurada a través de colecciones de documentos independientes:

- Usuario (Collection): Guarda los datos de cuentas, planes activos y perfiles.
- Reservas (Collection): Almacena el registro de citas de las clases.
- Horarios (Collection): Contiene los bloques horarios y entrenadores asignados.
- Planes (Collection): Almacena las características de las membresías ofrecidas.
- Asistencia (Collection): Registra el histórico de asistencia diaria de los alumnos.

**DINÁMICA DE DEPENDENCIAS Y FLUJO DEL SISTEMA:** El diagrama muestra gráficamente el flujo de trabajo mediante líneas de conexión: La capa de Frontend depende directamente de la capa de Servicios (indicado por la línea discontinua con flecha), lo que significa que ninguna página realiza acciones por sí misma, sino que invoca a la lógica de negocio correspondiente.

A su vez, la capa de Servicios se conecta a los servidores de Firebase (a través de líneas sólidas) para autenticar las peticiones en Firebase Authentication o para leer y escribir información en las colecciones de Firestore Database, cerrando así el ciclo completo de procesamiento de la aplicación.

# Diagrama de navegación



El diagrama de navegación detalla el flujo de experiencia de usuario y la estructura de pantallas del sistema GymOlympo. Muestra la ruta que sigue un usuario desde el acceso inicial, pasando por el proceso de autenticación, hasta dividirse en dos módulos principales diferenciados por el rol del usuario: Cliente y Administrador.

## MÓDULO DE INICIO Y AUTENTICACIÓN:

La navegación comienza en la pantalla de Inicio, la cual redirige de forma directa a la pantalla de Login (Inicio de Sesión). Desde el Login, el sistema permite rutas alternativas de gestión de cuenta representadas con líneas discontinuas:

- Registrarse: Para nuevos usuarios que desean crear una cuenta.
- Recuperar Contraseña: Para usuarios que necesitan restablecer sus credenciales de acceso.

Una vez que el usuario se autentica exitosamente, el sistema valida su rol y lo redirige a uno de los dos tableros principales (Dashboards).

## MODULO DEL CLIENTE (DASHBOARD CLIENTE):

Cuando un usuario accede con el rol de cliente, ingresa a su Dashboard principal, desde el cual puede navegar de manera independiente hacia las siguientes secciones y funciones:

- Ver Planes: Vista de las membresías disponibles.
- Ver Horarios: Consulta de la grilla horaria del gimnasio.
- Historial: Consulta de las actividades y asistencias pasadas.
- Perfil: Gestión de los datos personales del cliente.

Además, el dashboard del cliente incluye dos flujos transaccionales secuenciales:

- Flujo de Reserva: Desde la opción Reservar Sesión, el cliente pasa a Seleccionar Horario y finaliza el proceso en la pantalla de Confirmar Reserva.

- Flujo de Cancelación: Desde la opción Mis Reservas, el cliente puede optar por Cancelar Reserva, lo cual también lo lleva a la pantalla de Confirma Reserva para validar el cambio de estado.

#### MÓDULO DEL ADMINISTRADOR (DASHBOARD ADMINISTRADOR):

Cuando un usuario accede con credenciales de gestión, ingresa al Dashboard Administrador. Este módulo centraliza las herramientas de control operativo y de configuración del gimnasio a través de las siguientes opciones:

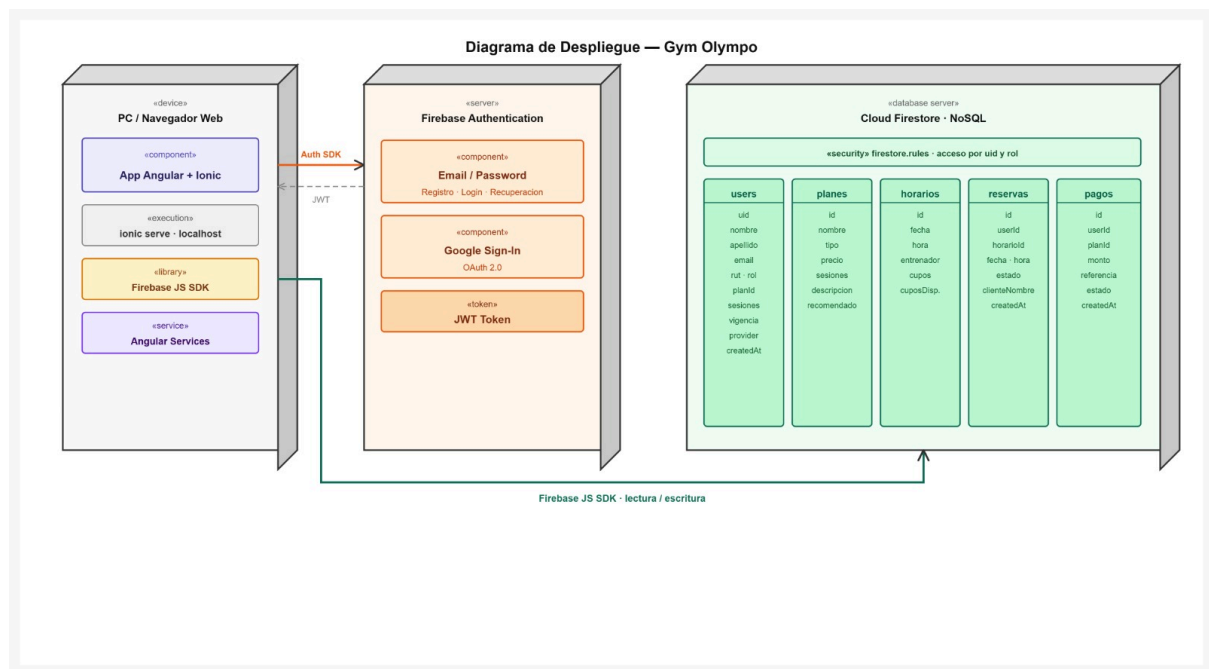
- Gestión de Clientes: Administración y visualización de los usuarios registrados.
- Registrar Asistencia: Control diario de la llegada de los alumnos.
- Reportes: Módulo de estadísticas e indicadores de rendimiento del gimnasio.
- Perfil Admin: Gestión de los datos del administrador actual.

Asimismo, cuenta con tres submódulos críticos organizados de forma jerárquica para el mantenimiento de datos:

- Submódulo Gestión Horarios: Despliega las opciones operativas de Crear Horarios, Editar Horarios y Eliminar Horario.
- Submódulo Gestion Planes: Permite la configuración comercial mediante las pantallas de Crear Plan y Editar Plan.
- Submódulo Gestión Reservas: Facilita el monitoreo y control de agendas a través de las opciones Ver Reservas y Eliminar Reserva.

En resumen, el sistema GymOlympo plantea una navegación intuitiva y segregada por perfiles, garantizando que el cliente tenga un acceso rápido a sus reservas y el administrador mantenga el control total de la operación en pocos clics.

# Diagrama de despliegue



El diagrama de despliegue muestra la arquitectura física, la infraestructura tecnológica y la distribución de los componentes de software del sistema GymOlympto. La solución adopta un enfoque moderno basado en la nube (Serverless) utilizando los servicios de Firebase, lo que elimina la necesidad de mantener un servidor físico propio y distribuye la carga entre el cliente y los servicios administrados de Google.

A continuación, se detallan los tres nodos principales y sus interacciones:

**NODO CLIENTE: PC / NAVEGADOR WEB (DEVICE):** Representa el entorno local del usuario o administrador desde donde se accede a la plataforma. Contiene las capas de ejecución del frontend:

- App Angular + Ionic (Component): Es la interfaz gráfica y lógica de la aplicación construida con el framework Angular junto a Ionic para asegurar un diseño adaptable y responsivo.
- Ionic serve - localhost (Execution): Especifica el entorno de ejecución en el navegador web local.
- Firebase JS SDK (Library): Librería oficial instalada en el cliente que permite la conexión directa y segura con los servicios en la nube de Firebase sin pasar por un backend intermedio.
- Angular Services (Service): Capa de servicios internos de Angular encargados de manejar la lógica de negocio en el frontend.

**NODO DE AUTENTICACIÓN: FIREBASE AUTHENTICATION (SERVER):** Es un servicio en la nube especializado encargado exclusivamente de la seguridad, el control de accesos y la gestión de identidades:

- Email / Password (Component): Componente para el flujo tradicional de login, registro y recuperación de cuentas mediante correo electrónico.

- Google Sign-In - OAuth 2.0 (Component): Componente que permite a los usuarios iniciar sesión de forma rápida y segura utilizando sus cuentas de Google mediante el protocolo OAuth 2.0.
- JWT Token (Token): Una vez que el usuario se autentica de forma correcta, este módulo genera y devuelve un token de seguridad firmado (JSON Web Token) que valida la identidad del usuario en las demás solicitudes.

**NODO DE BASE DE DATOS: CLOUD FIRESTORE - NOSQL (DATABASE SERVER):** Es el motor de persistencia de datos en la nube. Al ser una base de datos NoSQL orientada a documentos, almacena la información en colecciones estructuradas de la siguiente manera:

- Capa de Seguridad (firestore.rules): Controla el acceso a los datos en tiempo real mediante reglas de seguridad personalizadas. Filtra las consultas permitiendo lecturas y escrituras únicamente si el usuario cuenta con el UID (User ID) correcto y el rol adecuado (Cliente o Administrador).
- Colecciones del Sistema: Contiene los contenedores de datos analizados previamente en el diseño de base de datos: "users", "planes", "horarios", "reservas" y "pagos".

**FLUJO DE COMUNICACIÓN E INTERACCIONES:** El diagrama define dos canales de comunicación críticos desde el cliente hacia la nube:

- Canal de autenticación (Línea Naranja/Gris): El navegador web se conecta a Firebase Authentication usando el "Auth SDK" enviando las credenciales. Tras ser validadas, el servidor de autenticación responde devolviendo un "JWT" (Token) hacia el cliente de forma asíncrona.
- Canal de Datos (Línea Verde): El cliente se comunica directamente con la base de datos "Cloud Firestore" utilizando el "Firebase JS SDK" para realizar operaciones de lectura y escritura. Este canal es vigilado por la capa de reglas de seguridad ("firestore.rules"), la cual valida el token JWT del usuario antes de permitirle leer o modificar documentos de las colecciones (por ejemplo, al realizar una reserva o modificar un horario).

## Configuración de Entornos, Servidores y Continuidad

## Tecnologías utilizadas y justificación

### Frontend

El frontend está diseñado como una aplicación web híbrida moderna capaz de ejecutarse tanto en la web como empaquetarse en dispositivos móviles:

**Framework Principal: Angular (v17.0.2):** Es el framework central encargado de la lógica del cliente, enrutamiento, formularios reactivos y ciclo de vida de los componentes.

Framework de Interfaz de Usuario (UI): Ionic Framework (v7.0.0 / @ionic/angular): Proporciona los componentes visuales nativos y de alto rendimiento optimizados para móvil y web (botones, listas, modales, navegación, etc.). Utiliza "Icons" (v7.0.0) para la iconografía del sistema.

Contenedor Nativo / Despliegue Móvil: Capacitor (v5.7.4 / @capacitor/core): Permite empaquetar el frontend web compilado en aplicaciones móviles nativas (iOS y Android) y provee acceso a APIs nativas del dispositivo (teclado, barra de estado, respuesta háptica, etc.).

Lógica y Programación: TypeScript (v5.2.2): Todo el desarrollo de lógica del lado del cliente está tipado estáticamente con TypeScript.

Programación Reactiva: RxJS (~7.8.0): Utilizada para el manejo de flujos de datos asíncronos y eventos dentro de Angular.

Estilizado: SCSS (Sass): Hojas de estilo estructuradas y modulares en cada componente de la aplicación.

La selección de Angular, Ionic, Capacitor, TypeScript, RxJS, SCSS y las librerías complementarias responde a la necesidad de desarrollar una solución moderna, escalable y multiplataforma para GymOlympo. Estas tecnologías permiten construir una aplicación con una arquitectura organizada y mantenible, facilitando el desarrollo de nuevas funcionalidades y la evolución del sistema a largo plazo.

Angular proporciona una estructura sólida para el desarrollo de aplicaciones complejas, mientras que Ionic permite ofrecer una experiencia de usuario consistente tanto en dispositivos móviles como en navegadores web. Por su parte, Capacitor facilita la distribución de la aplicación en plataformas móviles sin necesidad de desarrollar versiones independientes, optimizando los recursos y tiempos de desarrollo.

El uso de TypeScript contribuye a mejorar la calidad del software mediante el tipado estático y la detección temprana de errores, mientras que RxJS permite gestionar eficientemente los procesos asíncronos y la comunicación con los servicios del sistema. Asimismo, SCSS aporta una mejor organización y reutilización de los estilos visuales, favoreciendo el mantenimiento y la personalización de la interfaz.

En conjunto, estas tecnologías fueron seleccionadas por su compatibilidad entre sí, su amplio soporte en la industria, su comunidad activa y su capacidad para satisfacer los requerimientos funcionales y no funcionales de GymOlympo, garantizando una solución eficiente, escalable y preparada para futuras mejoras.

## Backend

El proyecto sigue una arquitectura Serverless en la nube, donde Firebase actúa como el proveedor principal de Backend-as-a-Service (BaaS):

Base de Datos: Cloud Firestore:

- Base de datos NoSQL orientada a documentos y en tiempo real.
- Se utiliza para almacenar entidades de negocio tales como usuarios, reservas, clases, configuraciones del gimnasio, etc.

Cuenta con reglas de seguridad definidas en "firestore.rules" e índices en "firestore.indexes.json" para optimizar las consultas.

Autenticación: Firebase Authentication:

- Gestiona el registro, inicio de sesión y control de acceso.
- Implementa flujo de autenticación tradicional (correo y contraseña) y proveedores externos como Google (Google Sign-In).

## Base de datos / Backend as a Service

- Firebase (Firestore + Authentication + servicios asociados)

Firebase es utilizado en dos capacidades distintas y complementarias dentro del sistema. Firebase Authentication gestiona la identidad de los usuarios, proporcionando registro, inicio de sesión y emisión de tokens JWT seguros, siendo el único servicio de Firebase que interactúa directamente con el cliente. Firestore actúa como base de datos NoSQL en la nube, accesible exclusivamente desde el backend a través de Firebase Admin SDK, garantizando la integridad y seguridad de los datos de negocio. Esta separación de responsabilidades elimina el acceso directo a los datos desde el frontend, respetando los principios de seguridad y separación de responsabilidades de la arquitectura propuesta.

## Herramientas

- Github
- Trello
- Postman

Estas herramientas permiten gestionar el control de versiones del código, organizar las tareas del equipo mediante tableros Scrum y probar los endpoints de la API REST durante el desarrollo, alineándose con buenas prácticas de desarrollo de software.

## Tabla de Stack Tecnológico

| Elemento Tecnológico     | Nombre             | Justificación                                                                   | Factibilidad Técnica |
|--------------------------|--------------------|---------------------------------------------------------------------------------|----------------------|
| IDE                      | Visual Studio Code | Permite desarrollar aplicaciones Angular, Ionic y Firebase de manera eficiente. | Alta                 |
| Lenguaje de Programación | TypeScript         | Proporciona tipado estático y mayor mantenibilidad del código.                  | Alta                 |
| Framework Frontend       | Angular 17         | Permite desarrollar aplicaciones escalables y modulares.                        | Alta                 |

|                         |                   |                                                                                                        |      |
|-------------------------|-------------------|--------------------------------------------------------------------------------------------------------|------|
| Framework UI            | Ionic Framework 7 | Facilita el desarrollo multiplataforma para web y dispositivos móviles.                                | Alta |
| Gestor de Dependencias  | npm               | Permite administrar paquetes y librerías del proyecto.                                                 | Alta |
| Plataforma Cloud        | Firebase          | Proporciona servicios administrados de autenticación, almacenamiento y ejecución serverless.           | Alta |
| Ambiente y Contenedores | N/A               | El proyecto utiliza servicios administrados en la nube, por lo que no requiere contenedores dedicados. | Alta |
| Ambiente de Calidad     | Postman           | Permite validar y probar las funciones HTTPS implementadas.                                            | Alta |
| Base de Datos           | Cloud Firestore   | Base de datos NoSQL escalable y administrada por Firebase.                                             | Alta |
| Repositorio de Código   | GitHub            | Facilita el control de versiones y trabajo colaborativo.                                               | Alta |
| Gestión de Tareas       | Trello            | Permite organizar actividades y realizar seguimiento de los sprints.                                   | Alta |

## Viabilidad Técnica del Stack Tecnológico

La integración del stack tecnológico seleccionado es técnicamente viable debido a la compatibilidad existente entre Angular, Ionic y Firebase. Angular proporciona una estructura sólida para el desarrollo de aplicaciones web modernas, mientras que Ionic permite reutilizar el mismo código para dispositivos móviles y navegadores web.

Firebase complementa esta arquitectura mediante servicios administrados como Authentication y Firestore, eliminando la necesidad de administrar infraestructura propia y facilitando la escalabilidad del sistema. Asimismo, el uso de TypeScript en frontend y backend permite mantener consistencia tecnológica en todo el proyecto, reduciendo la complejidad del desarrollo y mantenimiento.

En conjunto, estas tecnologías permiten satisfacer los requerimientos funcionales y no funcionales definidos para GymOlympo, garantizando seguridad, disponibilidad, mantenibilidad y escalabilidad.

## Visión de la Situación Futura

Una vez implementada la solución, Gym Olympo contará con una plataforma digital que permitirá gestionar reservas, sesiones y pagos de forma centralizada y automatizada.

Los clientes podrán consultar horarios disponibles, reservar sesiones, gestionar sus pagos y revisar su historial desde cualquier dispositivo con acceso a internet. Por su parte, los administradores dispondrán de herramientas que facilitarán la gestión de clientes, planes y horarios, reduciendo significativamente la carga administrativa y los errores asociados a procesos manuales.

La implementación de la plataforma permitirá mejorar la organización interna del gimnasio, optimizar el uso de los recursos disponibles y entregar una mejor experiencia a los usuarios.

## Estrategia de Respaldo y Recuperación

La información almacenada en Cloud Firestore contará con mecanismos de respaldo proporcionados por la infraestructura de Firebase. Adicionalmente, se podrán realizar exportaciones periódicas de la base de datos para mantener copias de seguridad actualizadas.

En caso de fallas o pérdida de información, será posible restaurar los datos a partir de los respaldos disponibles, minimizando el impacto sobre la operación del sistema. Esta estrategia busca garantizar la continuidad operacional y la integridad de la información almacenada.

## Configuración del Ambiente de Pruebas

Con el propósito de validar el funcionamiento de la solución antes de su despliegue definitivo, se implementó un ambiente de pruebas orientado a simular las condiciones operativas del sistema dentro de un entorno controlado.

La configuración de este entorno permitió verificar la correcta integración entre los distintos componentes de la arquitectura propuesta, además de evaluar la estabilidad inicial de la plataforma durante la ejecución de funcionalidades críticas.

El ambiente de pruebas fue configurado utilizando las tecnologías y servicios cloud definidos dentro de la arquitectura del sistema. Para ello, se realizó la instalación y configuración de Node.js, Angular, Ionic Framework, dependencias del proyecto, servicios Firebase y herramientas necesarias para la ejecución y validación de la aplicación.

Dentro de este entorno se habilitó:

- Ejecución del frontend desarrollado con Angular e Ionic Framework.
- Configuración de Firebase Authentication para la gestión de usuarios.
- Integración con Cloud Firestore para la persistencia de información.
- Validación de la comunicación entre el frontend y los servicios serverless mediante funciones HTTPS.

Posteriormente, se realizaron pruebas funcionales y de integración sobre los principales módulos del sistema, incluyendo autenticación de usuarios, gestión de reservas, control de acceso, procesamiento de pagos, persistencia de datos y comunicación entre el frontend y los servicios de Firebase.

Estas pruebas permitieron detectar errores de integración de manera temprana, validar la estabilidad inicial del sistema y asegurar que las funcionalidades implementadas operaran correctamente dentro del entorno configurado.

Asimismo, el ambiente de pruebas permitió evidenciar el avance progresivo del proyecto mediante la validación continua de funcionalidades desarrolladas durante cada sprint de trabajo, garantizando el cumplimiento de los requerimientos funcionales y no funcionales definidos para GymOlympo.

## Respaldo y Validación de Datos

Como parte de las actividades de validación y aseguramiento del sistema, se realizaron pruebas orientadas a verificar la correcta persistencia y almacenamiento de información dentro de Cloud Firestore.

Estas pruebas permitieron comprobar que los datos generados desde la aplicación fueran almacenados correctamente dentro de la base de datos cloud y recuperados posteriormente sin pérdida de información.

La utilización de servicios cloud permitió mantener disponibilidad de la información y facilitar futuras tareas de respaldo, escalabilidad y mantenimiento del sistema.

Estas validaciones permitieron asegurar la integridad y consistencia de los datos procesados por la plataforma dentro del entorno de pruebas implementado.

## Servicios cloud y justificación

La solución se basa en una arquitectura cloud, utilizando distintos modelos de servicios que permiten el desarrollo, despliegue y operación del sistema de manera escalable, segura y eficiente.

### Modelo BaaS (Backend as a Service)

- Firebase Authentication
- Firebase Firestore
- Firebase Admin SDK (uso en backend)

Firebase se utiliza como plataforma Backend as a Service para la gestión de identidad y persistencia de datos. Firebase Authentication permite la administración segura de usuarios mediante registro, inicio de sesión y emisión de tokens JWT. Firestore funciona como base de datos NoSQL en la nube, accesible únicamente desde el backend para garantizar la seguridad e integridad de los datos. El Firebase Admin SDK permite la integración segura desde el servidor para la validación de tokens y acceso controlado a la información.

### Modelo SaaS (Software as a Service)

- GitHub
- Trello
- Postman

Estas herramientas se utilizan como servicios en la nube que apoyan el ciclo de desarrollo del software. GitHub permite el control de versiones del código, Trello facilita la gestión del trabajo mediante metodología Scrum, y Postman se utiliza para probar y validar las funciones HTTPS expuesta

## Beneficios del uso de Cloud Computing

El uso de servicios cloud permite al sistema:

- Escalabilidad automática según la demanda de usuarios y reservas del sistema.
- Alta disponibilidad del servicio, permitiendo acceso continuo 24/7.
- Reducción de costos asociados a infraestructura física y mantenimiento de servidores.
- Acceso remoto desde cualquier dispositivo con conexión a internet.
- Mayor flexibilidad en el despliegue, actualización y mantenimiento del sistema.

Además, la arquitectura cloud contribuye a la resiliencia del sistema, permitiendo una operación estable incluso ante variaciones en la carga de usuarios.

## Conceptualización de la solución

La solución propuesta consiste en el desarrollo de una plataforma digital orientada a la gestión de reservas, sesiones y asesorías del gimnasio Gym Olympo, con el objetivo de automatizar procesos manuales y mejorar la experiencia del usuario.

El sistema se fundamenta en una arquitectura cloud con separación de capas (frontend, backend e infraestructura), lo que permite una clara separación de responsabilidades entre sus componentes, garantizando escalabilidad, seguridad y mantenibilidad.

## Factibilidad de cumplimiento

La solución propuesta es técnicamente factible debido al uso de tecnologías cloud ampliamente adoptadas como Firebase, Angular e Ionic, las cuales permiten acelerar el desarrollo, reducir la complejidad de infraestructura y asegurar una implementación viable dentro de los tiempos y recursos del proyecto.

Asimismo, el uso de servicios administrados (BaaS y PaaS) reduce la necesidad de gestión de servidores físicos, permitiendo que el equipo se enfoque en la lógica de negocio y la experiencia del usuario.

## Propósito de la solución y atributos de calidad

### Propósito de la solución

El propósito de la solución es desarrollar una plataforma digital para el gimnasio Gym Olympo que permita automatizar la gestión de reservas, sesiones y asesorías, optimizando el control de horarios y cupos disponibles. Esto busca reemplazar los procesos manuales actuales por un sistema en línea más eficiente, accesible y seguro.

La solución permitirá a los clientes gestionar sus reservas de manera autónoma desde cualquier dispositivo con conexión a internet, mejorando la experiencia de usuario y reduciendo la carga administrativa del gimnasio.

## Valor agregado

La implementación del sistema genera valor al negocio mediante:

- Automatización de procesos manuales de agendamiento y control de sesiones.
- Digitalización del sistema de reservas y asesorías.
- Optimización en la gestión de horarios y control de cupos.
- Mejora en la administración y disponibilidad de la información.
- Reducción de errores asociados a procesos manuales.

## Atributos de calidad

El sistema estará diseñado considerando los siguientes atributos de calidad:

- Integridad: consistencia y exactitud de los datos almacenados y procesados.
- Confiabilidad: funcionamiento estable del sistema bajo diferentes condiciones de uso.
- Usabilidad: interfaz intuitiva y accesible que facilita la interacción de los usuarios.
- Disponibilidad: acceso a la información de reservas y horarios en tiempo real.
- Seguridad: protección de datos mediante autenticación y control de acceso basado en tokens.

## Plan de Calidad

### Definiciones de Calidad del Proyecto

Para asegurar la calidad del software desarrollado, se establecen los siguientes criterios:

- Cumplimiento de los requerimientos funcionales y no funcionales definidos.
- Correcta integración entre frontend, backend y servicios cloud.
- Código fuente documentado y mantenible.
- Validación mediante pruebas funcionales e integración.
- Control de versiones mediante GitHub.
- Revisión continua del avance durante cada sprint.

Una funcionalidad será considerada terminada cuando haya sido desarrollada, probada, documentada y validada por el equipo de trabajo.

## Casos de pruebas

|                                                   |       |            |
|---------------------------------------------------|-------|------------|
| Registro de usuario nuevo con Correo y Contraseña | CP-01 | ¿Prueba de |
|---------------------------------------------------|-------|------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |  |                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|----------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |  | despliegue? No |
| <b>Descripción:</b> Verificar que un nuevo usuario pueda registrarse exitosamente completando el formulario de registro con datos válidos.                                                                                                                                                                                                                                                                                                                                                                                                                                                            |  |                |
| <b>Prerrequisitos:</b> 1. Firebase Authentication habilitado. 2. Firestore configurado con colección usuarios. 3. Aplicación en ejecución (ionic serve o APK). 4. Correo electrónico válido disponible. 5. No tener una sesión activa en el navegador. 6. El correo ingresado no debe estar registrado en el sistema.                                                                                                                                                                                                                                                                                 |  |                |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Ingresar a la aplicación y ser redirigido a la página de Login (/login).</li> <li>2. Hacer clic en el enlace "¿No tienes cuenta? Regístrate".</li> <li>3. Ingresar un RUT válido (ej. 12.345.678-9).</li> <li>4. Ingresar Nombre (Juan) y Apellido (Pérez).</li> <li>5. Ingresar un número telefónico (ej. +56 9 1234 5678).</li> <li>6. Ingresar un correo electrónico (juan.perez@example.com).</li> <li>7. Ingresar Contraseña (Password123) y confirmar la misma contraseña.</li> <li>8. Hacer clic en el botón "Registrarse".</li> </ol> |  |                |
| <b>Resultado esperado:</b><br>La cuenta se crea correctamente en Firebase Auth.<br>Se almacena el perfil en la colección users de Firestore con rol 'cliente'.<br>El usuario es redirigido automáticamente a la página principal de cliente (/main).                                                                                                                                                                                                                                                                                                                                                  |  |                |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |  |                |

|                                                                                                                                                                                                                                                                                                        |              |                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|---------------------------|
| <b>Inicio de sesión con Correo y Contraseña válidos</b>                                                                                                                                                                                                                                                | <b>CP-02</b> | ¿Prueba de despliegue? No |
| <b>Descripción:</b> Verificar que un usuario registrado pueda acceder a la aplicación ingresando sus credenciales válidas.                                                                                                                                                                             |              |                           |
| <b>Prerrequisitos:</b> 1. Usuario previamente registrado en el sistema. 2. Firebase Authentication habilitado. 3. Aplicación en ejecución.                                                                                                                                                             |              |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Ir a la pantalla de inicio de sesión (/login).</li> <li>2. Ingresar el correo electrónico (juan.perez@example.com).</li> <li>3. Ingresar la contraseña correcta (Password123).</li> <li>4. Hacer clic en el botón "Iniciar Sesión".</li> </ol> |              |                           |
| <b>Resultado esperado:</b> El sistema valida las credenciales a través de Firebase Auth. Se obtiene un token JWT. Se redirige al dashboard del cliente o administrador según el rol asignado.                                                                                                          |              |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                                     |              |                           |

|                                                                                                                                              |              |                           |
|----------------------------------------------------------------------------------------------------------------------------------------------|--------------|---------------------------|
| <b>Inicio de sesión fallido por contraseña incorrecta</b>                                                                                    | <b>CP-03</b> | ¿Prueba de despliegue? No |
| <b>Descripción:</b> Validar que el sistema impida el ingreso con credenciales erróneas e informe al usuario mediante un mensaje en pantalla. |              |                           |
| <b>Prerrequisitos:</b> 1. Tener un usuario registrado previamente.                                                                           |              |                           |

|                                                                                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Ir a la pantalla de inicio de sesión (/login).</li> <li>2. Ingresar el correo electrónico (juan.perez@example.com).</li> <li>3. Ingresar una contraseña incorrecta (ej. WrongPass123).</li> <li>4. Hacer clic en el botón "Iniciar Sesión".</li> </ol> |
| <b>Resultado esperado:</b><br>La aplicación rechaza la autenticación de Firebase.<br>Se despliega el mensaje de error en pantalla: "Correo o contraseña incorrectos".<br>El spinner de carga se detiene y los campos permanecen habilitados.                                                                   |
| <b>Resultado obtenido:</b> Exitoso. El bloque try-catch captura el error de Firebase Auth, detiene el estado de carga y muestra el mensaje de credenciales incorrectas.                                                                                                                                        |

| Validación de formato de RUT en Registro                                                                                                                                                                                                                                                | CP-04 | ¿Prueba de despliegue? No |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Comprobar que el formulario de registro impida el envío y notifique al usuario si el RUT ingresado no posee el formato estándar chileno (XX.XXX.XXX-X).                                                                                                             |       |                           |
| <b>Prerrequisitos:</b> 1. Estar en la página de Registro (/register).                                                                                                                                                                                                                   |       |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Escribir un RUT incorrecto o incompleto (ej. 123456789 o 12.345.67-8) en el campo de RUT.</li> <li>2. Rellenar el resto de los campos con datos de prueba válidos.</li> <li>3. Hacer clic en el botón "Registrarse".</li> </ol> |       |                           |
| <b>Resultado esperado:</b><br>El formulario bloquea el envío de la información.<br>El campo del RUT se resalta en rojo indicando un error de validación.<br>Se visualiza el mensaje "El RUT ingresado no es válido".                                                                    |       |                           |
| <b>Resultado obtenido:</b> Exitoso. El validador reactivo (validarRut) marca el formulario como inválido y previene la llamada al método de registro.                                                                                                                                   |       |                           |

| Inicio de sesión con Google y Completado de Perfil                                                                                                                                                                                                                                                                                                                                                       | CP-05 | ¿Prueba de despliegue? No |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Validar el acceso mediante la cuenta de Google y el flujo obligatorio de completar el perfil de usuario (RUT y Teléfono) si se ingresa por primera vez.                                                                                                                                                                                                                              |       |                           |
| <b>Prerrequisitos:</b> 1. Disponer de una cuenta Google activa no registrada previamente en la plataforma.                                                                                                                                                                                                                                                                                               |       |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. En la página de Login (/login), presionar el botón "Continuar con Google".</li> <li>2. Seleccionar la cuenta Google en la ventana emergente de Google.</li> <li>3. Al ser redirigido automáticamente a /completar-perfil por ser cuenta nueva, rellenar los datos de RUT y Teléfono.</li> <li>4. Presionar el botón "Guardar Perfil".</li> </ol> |       |                           |
| <b>Resultado esperado:</b><br>La cuenta se vincula correctamente y se crea su perfil en Firestore con rol 'cliente'.<br>Tras presionar "Guardar Perfil", los campos RUT y Teléfono se actualizan en Firestore.                                                                                                                                                                                           |       |                           |

|                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| El usuario es redirigido a /main para navegar normalmente.                                                                                                                                                          |
| <b>Resultado obtenido:</b> Exitoso. El sistema detecta el perfil incompleto a través del método isProfileComplete(), bloquea el menú de navegación y obliga a completar los datos antes de conceder acceso a /main. |

| Actualización de datos y cambio de contraseña desde el Perfil                                                                                                                                                                                                                                                                                    | CP-06 | ¿Prueba de despliegue? No |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el usuario pueda modificar sus datos personales y actualizar su contraseña desde la página de perfil.                                                                                                                                                                                                          |       |                           |
| <b>Prerrequisitos:</b> 1. Sesión de cliente activa. Estar en la página de Perfil (/perfil).                                                                                                                                                                                                                                                      |       |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Modificar el campo Nombre a Juan Carlos y el campo Teléfono a un nuevo número.</li> <li>2. En el campo "Nueva Contraseña", escribir NewPassword123.</li> <li>3. En el campo "Confirmar Contraseña", escribir NewPassword123.</li> <li>4. Presionar el botón "Guardar Perfil".</li> </ol> |       |                           |
| <b>Resultado esperado:</b><br>El documento del usuario en Firestore actualiza los campos Nombre y Teléfono.<br>Firebase Auth actualiza la contraseña del usuario.<br>Se muestra el mensaje "Perfil actualizado correctamente".                                                                                                                   |       |                           |
| <b>Resultado obtenido:</b> Exitoso. Los datos se actualizan en Firestore y la contraseña se modifica de manera segura en Firebase Auth. Se limpian los campos de contraseña.                                                                                                                                                                     |       |                           |

| Cierre de sesión                                                                                                                                                                                                                                                   | CP-07 | ¿Prueba de despliegue? No |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Validar que el usuario pueda finalizar su sesión activa de forma segura.                                                                                                                                                                       |       |                           |
| <b>Prerrequisitos:</b> 1. Sesión activa de cliente o administrador.                                                                                                                                                                                                |       |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Abrir la página de perfil (/perfil).</li> <li>2. Hacer clic en el botón "Cerrar Sesión" (o icono correspondiente).</li> <li>3. En la alerta emergente de confirmación, hacer clic en "Aceptar".</li> </ol> |       |                           |
| <b>Resultado esperado:</b><br>La sesión se cierra en Firebase Auth.<br>El usuario es redirigido a la pantalla pública de Login (/login).<br>Al intentar volver atrás en el navegador, se impide el acceso a /main o /perfil.                                       |       |                           |
| <b>Resultado obtenido:</b> Exitoso. El método logout() de UserService destruye la sesión y el enrutador redirige a /login. El guard de autenticación (AuthGuard) bloquea accesos posteriores.                                                                      |       |                           |

| Recuperación de contraseña por email                                                                                                                           | CP-08 | ¿Prueba de despliegue? No |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el sistema envía correctamente un correo de recuperación de contraseña al email registrado mediante Firebase Authentication. |       |                           |

|                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prerrequisitos:</b> 1. Usuario registrado con email válido. 2. Acceso al correo electrónico del usuario. 3. Firebase Authentication habilitado.                                                                                                               |
| <b>Pasos:</b> 1. Navegar a la pantalla de Login. 2. Presionar '¿Olvidaste tu contraseña?'. 3. Ingresar el email registrado. 4. Presionar Enviar. 5. Revisar la bandeja de entrada del correo. 6. Hacer clic en el enlace recibido y establecer nueva contraseña. |
| <b>Resultado esperado:</b> El sistema envía un email de recuperación. El usuario puede acceder al enlace y establecer una nueva contraseña. El inicio de sesión con la nueva contraseña es exitoso.                                                              |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                               |

| Registro y gestión de clientes por administrador                                                                                                                                                                                                                                                                                          | CP-09 | ¿Prueba de despliegue? No |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el administrador puede crear, visualizar, editar y asignar planes a los clientes del gimnasio desde el módulo de administración.                                                                                                                                                                        |       |                           |
| <b>Prerrequisitos:</b> 1. Usuario con rol de administrador autenticado. 2. Colección usuarios configurada en Firestore. 3. Planes de entrenamiento disponibles en el sistema.                                                                                                                                                             |       |                           |
| <b>Pasos:</b> 1. Autenticarse como administrador. 2. Navegar al módulo de Gestión de Clientes. 3. Crear un nuevo cliente con datos completos (nombre, RUT, email, teléfono). 4. Asignar un plan de 12 sesiones en 4 semanas. 5. Guardar y verificar en Firestore. 6. Editar el plan asignado. 7. Verificar la actualización en Firestore. |       |                           |
| <b>Resultado esperado:</b> El cliente es creado y guardado correctamente en Firestore. El plan queda asociado al cliente. Las ediciones se reflejan en tiempo real. El cliente puede verse en el listado de clientes.                                                                                                                     |       |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                                                                        |       |                           |

| Visualización de Planes y Filtros (Presencial / Online)                                                                                                                                                                                                                                                                         | CP-10 | ¿Prueba de despliegue? No |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Validar que los clientes puedan explorar los planes ofertados organizados por tipo (Clase Presencial o Asesoría Online).                                                                                                                                                                                    |       |                           |
| <b>Prerrequisitos:</b> 1. Sesión de cliente activa. Navegar a la página de Planes (/planes).                                                                                                                                                                                                                                    |       |                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Observar la lista de planes iniciales en pantalla (por defecto Clases Presenciales).</li> <li>2. Hacer clic en el tab/filtro de "Asesoría Online".</li> <li>3. Observar los planes listados.</li> <li>4. Cambiar nuevamente al tab de "Clases Presenciales".</li> </ol> |       |                           |
| <b>Resultado esperado:</b><br>En "Clases Presenciales" se visualizan los planes de entrenamiento presencial (ej: "Prolympo", "Empieza a moverte").<br>En "Asesoría Online" se visualizan las opciones de planes de asesoría remota (ej: Asesoría de 1, 2 o 3 meses).                                                            |       |                           |
| <b>Resultado obtenido:</b> Exitoso. El filtrado reactivo de Angular en planesFiltrados segmenta la lista en base a la propiedad tipo de cada plan de manera inmediata.                                                                                                                                                          |       |                           |

|                                            |       |  |
|--------------------------------------------|-------|--|
| Selección de Plan y Aceptación de Términos | CP-11 |  |
|--------------------------------------------|-------|--|

|                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descripción:</b> Comprobar que al seleccionar un plan se despliegue una ventana de términos y condiciones antes de iniciar el pago.                                                                                                                                                                                                  |
| <b>Prerrequisitos:</b> 1. Sesión activa de cliente. No tener un plan activo vigente con sesiones disponibles.                                                                                                                                                                                                                           |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Navegar a /planes.</li> <li>2. Seleccionar el plan "Prolympo" presionando el botón "Comprar".</li> <li>3. Revisar el cuadro de diálogo (alerta de Ionic) con los términos detallados de vigencia y reembolsos.</li> <li>4. Presionar la opción "Acepto y continuar".</li> </ol> |
| <b>Resultado esperado:</b><br>Se genera un registro en la colección pagos de Firestore en estado 'pendiente'.<br>La alerta se cierra y el usuario es redirigido automáticamente a la página de Pagos (/pagos).                                                                                                                          |
| <b>Resultado obtenido:</b> Exitoso. El sistema agrega el registro de pago asociado al usuario, asignando el estado "pendiente", e inicia la navegación automática.                                                                                                                                                                      |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |              |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--|
| <b>Pago Exitoso con Tarjeta de Crédito Simulada</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <b>CP-12</b> |  |
| <b>Descripción:</b> Verificar que el cliente pueda realizar el pago simulado de su plan ingresando datos válidos de tarjeta de crédito.                                                                                                                                                                                                                                                                                                                                                                              |              |  |
| <b>Prerrequisitos:</b> 1. Tener un pago en estado "pendiente" en la vista /pagos.                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |  |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Navegar a /pagos.</li> <li>2. En el plan pendiente, presionar el botón "Pagar".</li> <li>3. Rellenar los campos del modal de tarjeta:               <ul style="list-style-type: none"> <li>* Número de tarjeta: 1234 5678 1234 5678 (16 dígitos).</li> <li>* Nombre del titular: Juan Pérez.</li> <li>* Fecha de vencimiento: 12/30 (formato MM/AA).</li> <li>* CVV: 123 (3 dígitos).</li> </ul> </li> <li>4. Presionar el botón "Pagar [Monto]".</li> </ol> |              |  |
| <b>Resultado esperado:</b><br>Se muestra el estado de procesamiento por 2 segundos.<br>Transcurridos los 2 segundos, se muestra el estado de éxito con un número de referencia (ej: MOCK-WP-XXXXXX).<br>El estado del pago cambia a 'validado' en Firestore.<br>El perfil del usuario actualiza los campos: planActivo con el nombre del plan, sesionesDisponibles y sesionesTotales según el plan, y calcula la vigencia (vigenciaFin a 1 mes desde hoy).<br>El modal se cierra automáticamente.                    |              |  |
| <b>Resultado obtenido:</b> Exitoso. La transacción en Firestore asegura que la membresía se active sincronizadamente con la aprobación del pago. El usuario visualiza la membresía activa en /main.                                                                                                                                                                                                                                                                                                                  |              |  |

|                                                                                                                                                                     |              |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--|
| <b>Validación de campos en pasarela de Pago</b>                                                                                                                     | <b>CP-13</b> |  |
| <b>Descripción:</b> Probar las restricciones de entrada de datos en la simulación de tarjeta de crédito para evitar transacciones con datos incompletos o erróneos. |              |  |

|                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prerrequisitos:</b> 1. Modal de pago abierto para un plan pendiente en /pagos.                                                                                                                                                                                                                                       |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Intentar pagar dejando el número de tarjeta en 1234 5678 (8 dígitos).</li> <li>2. Intentar pagar con un titular vacío.</li> <li>3. Escribir un mes inválido en vencimiento (ej. 15/29).</li> <li>4. Dejar el CVV con 2 dígitos (ej. 12).</li> </ol>             |
| <b>Resultado esperado:</b><br>El sistema muestra alertas nativas de error específicas impidiendo el envío:<br>"El número de tarjeta debe tener exactamente 16 dígitos."<br>"Por favor, ingresa el nombre del titular."<br>"El mes de vencimiento debe ser entre 01 y 12."<br>"El CVV debe tener exactamente 3 dígitos." |
| <b>Resultado obtenido:</b> Exitoso. El componente valida cada una de las condiciones mediante expresiones regulares y bloques condicionales antes de iniciar la simulación de red.                                                                                                                                      |

|                                                                                                                                                                                                                                                                           |              |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--|
| <b>Eliminación de Pago Pendiente</b>                                                                                                                                                                                                                                      | <b>CP-14</b> |  |
| <b>Descripción:</b> Verificar que un cliente pueda descartar una solicitud de pago pendiente si decide cambiar de opinión.                                                                                                                                                |              |  |
| <b>Prerrequisitos:</b> 1. Tener un registro de pago en estado "pendiente" visible en /pagos.                                                                                                                                                                              |              |  |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Ir a /pagos.</li> <li>2. Ubicar el plan pendiente y presionar el botón con el ícono de papelera ("Eliminar").</li> <li>3. Confirmar la acción en la ventana de confirmación del navegador ("Aceptar").</li> </ol> |              |  |
| <b>Resultado esperado:</b><br>El documento de pago pendiente es eliminado de Firestore.<br>El listado se actualiza inmediatamente en pantalla y desaparece el plan.<br>Se muestra el mensaje: "Plan pendiente eliminado correctamente."                                   |              |  |
| <b>Resultado obtenido:</b> Exitoso. La lógica de eliminarPagoPendiente() de gym.service.ts valida que el pago pertenezca al usuario con sesión iniciada y que esté en estado "pendiente" antes de realizar el borrado.                                                    |              |  |

|                                                                                                                                                                                                                           |              |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--|
| <b>Reservas de Clases</b>                                                                                                                                                                                                 | <b>CP-15</b> |  |
| <b>Descripción:</b> Comprobar que se muestren únicamente los bloques horarios que cuentan con cupos disponibles para una fecha seleccionada.                                                                              |              |  |
| <b>Prerrequisitos:</b> 1. Existir horarios creados en el sistema para una fecha futura.                                                                                                                                   |              |  |
| <b>Pasos:</b> <ol style="list-style-type: none"> <li>1. Navegar a la página de Reservas (/reservas).</li> <li>2. Hacer clic en el selector de fecha y elegir un día que tenga horarios agendados (ej: mañana).</li> </ol> |              |  |

**Resultado esperado:**

Se despliega una lista o cuadrícula con los horarios del día.

Solo se muestran los horarios que tienen cuposDisponibles > 0.

**Resultado obtenido:** Exitoso. El observable de horarios filtra dinámicamente los registros en base a la fecha y a la disponibilidad de cupos mayores a cero.

**Creación de Reserva Exitosa****CP-16**

**Descripción:** Validar que un usuario con plan activo y sesiones disponibles pueda agendar un bloque de entrenamiento.

**Prerrequisitos:** 1. Sesión activa de cliente.

2. El perfil del usuario debe tener un plan activo y al menos 1 sesión disponible (sesionesDisponibles >= 1).

3. Estar en la página /reservas y haber seleccionado un día y bloque horario.

**Pasos:**

1. Seleccionar una fecha válida con horarios disponibles.

2. Hacer clic en un bloque horario disponible (ej. 18:00).

3. Hacer clic en el botón "Confirmar Reserva".

**Resultado esperado:**

Se muestra el mensaje "Reserva confirmada correctamente".

El número de sesionesDisponibles del usuario se reduce en 1 en Firestore.

El número de cuposDisponibles del bloque horario se reduce en 1 en Firestore.

Se crea un registro en la colección reservas con estado 'confirmada'.

**Resultado obtenido:** Exitoso. La transacción de Firestore (runTransaction) asegura la atomicidad del descuento del cupo horario y del descuento de la sesión del usuario al guardar la reserva.

**Restricción de Reserva sin Plan Activo****CP-17**

**Descripción:** Verificar que la aplicación impida reservar a un usuario que no tiene una membresía contratada o validada.

**Prerrequisitos:** 1. Cuenta de usuario nueva o con plan vencido (planActivo === "").

**Pasos:**

1. Iniciar sesión con un usuario sin plan activo.

2. Ir a la vista de Reservas (/reservas).

3. Seleccionar una fecha e intentar presionar un bloque de horario.

**Resultado esperado:**

El sistema muestra un mensaje de advertencia en rojo indicando "Debes tener un plan activo para reservar".  
No se permite seleccionar bloques horarios ni confirmar reservas.

**Resultado obtenido:** Exitoso. La vista inhabilita la interacción con los bloques y muestra el mensaje respectivo utilizando `tienePlanActivo()`.

**Restricción de Reserva sin Sesiones Disponibles****CP-18**

**Descripción:** Validar que no se permita agendar si el usuario ya consumió todas las sesiones de su plan.

**Prerrequisitos:** 1. Usuario con plan contratado, pero con `sesionesDisponibles`  $\neq$  0.

**Pasos:**

1. Iniciar sesión con el usuario correspondiente.
2. Navegar a la página de Reservas (/reservas).
3. Seleccionar una fecha y presionar en un horario.
4. Intentar presionar "Confirmar Reserva".

**Resultado esperado:**

Se visualiza un mensaje en pantalla indicando "No tienes sesiones disponibles".  
El proceso de reserva queda bloqueado y no se guarda ningún registro.

**Resultado obtenido:** Exitoso. El código del componente controla que `sesionesDisponibles` sea mayor a cero, bloqueando el guardado en caso de ser cero.

**Cancelación de Reserva por el Cliente (Pérdida de Sesión)****CP-19**

**Descripción:** Verificar que la cancelación de una reserva por parte del usuario libere el cupo de la clase pero no le reembolse la sesión (política del gimnasio).

**Prerrequisitos:** 1. Tener al menos una reserva activa y confirmada en /reservas.

**Pasos:**

1. Ir a /reservas.
2. En el listado de reservas activas, hacer clic en "Cancelar" en una de ellas.
3. Leer el mensaje de advertencia: "Si cancelas esta reserva perderás la sesión utilizada. Esta acción no puede deshacerse."
4. Presionar el botón "Cancelar reserva" en el diálogo.

**Resultado esperado:**

El estado de la reserva cambia a 'cancelada' en Firestore.

El horario correspondiente incrementa sus cuposDisponibles en 1 en Firestore.

Las sesionesDisponibles del usuario NO se incrementan (permanecen iguales).

Se muestra el mensaje "Reserva cancelada correctamente".

**Resultado obtenido:** Exitoso. La transacción realiza las actualizaciones correspondientes en la reserva y en el horario, y respeta la regla de no reembolsar sesiones por inasistencias o cancelaciones.

| Restricción de Acceso a Administración (Guardia de Ruta)                                                                                                                                                          | CP-20 |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--|
| <b>Descripción:</b> Comprobar que un cliente común no pueda ingresar a las rutas exclusivas de administración escribiendo la URL manualmente.                                                                     |       |  |
| <b>Prerrequisitos:</b> 1. Sesión de cliente activa (rol: 'cliente').                                                                                                                                              |       |  |
| <b>Pasos:</b> <ol style="list-style-type: none"><li>1. Iniciar sesión como un cliente normal.</li><li>2. Escribir manualmente la URL /admin en la barra de direcciones del navegador y presionar Enter.</li></ol> |       |  |
| <b>Resultado esperado:</b> <p>La ruta es bloqueada por el AdminGuard.</p> <p>El cliente es redirigido a la página de inicio o login, impidiendo el renderizado del panel administrativo.</p>                      |       |  |
| <b>Resultado obtenido:</b> Exitoso. El guard del enrutador de Angular consulta el perfil en Firestore y, al no tener rol "admin", cancela la navegación y redirige.                                               |       |  |

| Carga de KPIs y Alertas en Dashboard de Administrador                                                                                                                                                    | CP-21 |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--|
| <b>Descripción:</b> Validar que al ingresar al panel de administración se calculen y muestren correctamente los KPIs principales y las alertas del sistema.                                              |       |  |
| <b>Prerrequisitos:</b> 1. Iniciar sesión como administrador (rol: 'admin').                                                                                                                              |       |  |
| <b>Pasos:</b> <ol style="list-style-type: none"><li>1. Ingresar a /admin.</li><li>2. Observar las tarjetas de KPIs en la parte superior.</li><li>3. Revisar el panel de "Alertas del Sistema".</li></ol> |       |  |

**Resultado esperado:**

Los KPIs cargan en tiempo real: total de reservas del día, cantidad de pagos pendientes, clientes con plan activo, etc. En el panel de alertas se muestran advertencias relevantes (ej: pagos pendientes antiguos, planes por expirar).

**Resultado obtenido:** Exitoso. Las funciones auxiliares en admin.utils.ts procesan adecuadamente las colecciones y muestran tarjetas detalladas e intuitivas.

| Confirmación de Asistencia (Check-in) por el Admin                                                                                                                                                                                                                               | CP-22 |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--|
| <b>Descripción:</b> Verificar que el administrador pueda registrar la asistencia física del cliente a la clase agendada.                                                                                                                                                         |       |  |
| <b>Prerrequisitos:</b> 1. Sesión de administrador activa. Que existan reservas para el día de hoy en la tabla "Reservas Recientes".                                                                                                                                              |       |  |
| <b>Pasos:</b> <ol style="list-style-type: none"><li>1. Ubicar la reserva de un cliente en la tabla de reservas de hoy en /admin.</li><li>2. Hacer clic en el botón "Confirmar Asistencia" o "Check-in" correspondiente a esa reserva.</li></ol>                                  |       |  |
| <b>Resultado esperado:</b> <p>El botón de la reserva se deshabilita temporalmente.</p> <p>Se actualiza el campo checkIn con la marca de tiempo actual y el estado queda validado en la base de datos.</p> <p>Se visualiza un mensaje: "Asistencia registrada correctamente".</p> |       |  |
| <b>Resultado obtenido:</b> Exitoso. El método confirmarAsistencia(reservaId) ejecuta una transacción en Firestore que marca la reserva con la fecha de check-in correspondiente.                                                                                                 |       |  |

| Cancelación de Reserva por el Administrador                                                                                                                                                                                                            | CP-23 |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--|
| <b>Descripción:</b> Verificar que el administrador pueda dar de baja la reserva de un cliente desde el dashboard principal.                                                                                                                            |       |  |
| <b>Prerrequisitos:</b> 1. Sesión de administrador activa.                                                                                                                                                                                              |       |  |
| <b>Pasos:</b> <ol style="list-style-type: none"><li>1. Ir a /admin.</li><li>2. Seleccionar una reserva activa en la tabla de reservas recientes.</li><li>3. Hacer clic en el botón "Cancelar" y confirmar la acción en el cuadro de diálogo.</li></ol> |       |  |

**Resultado esperado:**

Se procesa la cancelación en Firestore, liberando el cupo del horario asignado.  
La interfaz se actualiza de inmediato para reflejar el cambio.

**Resultado obtenido:** Exitoso. La reserva pasa a estado "cancelada", el horario recupera su cupo disponible, y se muestra un Toast de confirmación de éxito.

**Generación Automática de Horarios para 30 Días****CP-24**

**Descripción:** Validar que el administrador pueda crear en lote los horarios por defecto para el mes entrante.

**Prerrequisitos:** 1. Sesión de administrador activa. Estar en la página de administración /admin.

**Pasos:**

1. Desplazarse hasta la sección de "Acciones Rápidas".
2. Presionar el botón "Generar Horarios 30 Días".

**Resultado esperado:**

El sistema procesa un bucle para los próximos 30 días, omitiendo fechas ya creadas.  
Se insertan los bloques de horarios estándar (ej: 08:00, 09:00, 18:00, 19:00) con sus cupos por defecto (3 cupos) y entrenador asignado en Firestore.  
Se visualiza un Toast de éxito en verde.

**Resultado obtenido:** Exitoso. El método generarHorariosProximos30Dias() inserta las entradas de forma correcta en la colección horarios de Firestore sin duplicar los horarios preexistentes.

**Creación de Plan Personalizado (Custom Plan)****CP-25**

**Descripción:** Validar que el administrador pueda dar de alta un plan personalizado con sus características y detalles particulares.

**Prerrequisitos:** 1. Sesión de administrador activa. Navegar al subpanel de gestión de planes (/admin/planes).

**Pasos:**

1. Rellenar el formulario de nuevo plan:
  - \* Nombre: Plan Fuerza Olimpo.
  - \* Precio: 35000.
  - \* Tipo: clase.

|                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>* Sesiones: 10.</p> <p>2. Añadir características en el campo de descripción haciendo clic en "Agregar" (ej: "Acceso ilimitado a zona de peso libre", "Evaluación bioimpedancia").</p> <p>3. Hacer clic en "Guardar Plan".</p>   |
| <p><b>Resultado esperado:</b></p> <p>El plan es ingresado en la colección planes de Firestore y se limpia el formulario.</p> <p>Si un cliente accede a la vista de planes, el nuevo plan aparece disponible para contratación.</p> |
| <p><b>Resultado obtenido:</b> Exitoso. El plan se guarda en Firestore, se notifica mediante un Toast de éxito y el formulario se reinicia de manera limpia.</p>                                                                    |

| Eliminación de Plan Personalizado                                                                                                                                                                                                                                                   | CP-26 |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--|
| <b>Descripción:</b> Validar que el administrador pueda remover de forma definitiva un plan personalizado creado anteriormente.                                                                                                                                                      |       |  |
| <b>Prerrequisitos:</b> 1. Existir al menos un plan personalizado creado en la colección planes de Firestore.                                                                                                                                                                        |       |  |
| <p><b>Pasos:</b></p> <ol style="list-style-type: none"> <li>1. Ir a /admin/planes.</li> <li>2. En el listado de planes personalizados inferiores, hacer clic en "Eliminar" en el plan "Plan Fuerza Olimpo".</li> <li>3. Confirmar la eliminación en la alerta emergente.</li> </ol> |       |  |
| <p><b>Resultado esperado:</b></p> <p>El documento del plan es eliminado de la colección planes de Firestore.</p> <p>Desaparece del listado de planes del administrador y de la vista de los clientes.</p>                                                                           |       |  |
| <b>Resultado obtenido:</b> Exitoso. Se remueve de la base de datos de manera definitiva tras el clic y la confirmación del administrador.                                                                                                                                           |       |  |

| Gestión de agenda y control de cupos                                                                                                                                                                                                                                                                                                                                  | CP-26 | ¿Prueba de despliegue? No |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el administrador puede gestionar la agenda de sesiones, configurar horarios, y que el sistema limita automáticamente los cupos por horario, bloqueando recuperaciones fuera de plazo.                                                                                                                                               |       |                           |
| <b>Prerrequisitos:</b> 1. Administrador autenticado. 2. Horarios configurados en Firestore. 3. Clientes con planes activos registrados.                                                                                                                                                                                                                               |       |                           |
| <b>Pasos:</b> 1. Ingresar como administrador. 2. Navegar al módulo Agenda. 3. Crear o modificar un horario con cupo máximo definido. 4. Reservar el horario hasta completar el cupo. 5. Intentar agregar un cliente adicional al horario lleno. 6. Verificar mensaje de error por sobrecupo. 7. Intentar agendar una sesión fuera del período contratado del cliente. |       |                           |

|                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resultado esperado:</b> El sistema muestra el horario con cupos disponibles. Al llegar al límite, bloquea nuevas reservas e informa al usuario. Las sesiones fuera del período no son permitidas. |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                   |

| Validación de sesiones dentro del período contratado                                                                                                                                                                                                                                  | CP-27 | ¿Prueba de despliegue? No |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el sistema valida que las sesiones sean utilizadas únicamente dentro del período contratado (4 semanas) y rechaza intentos de uso fuera de plazo.                                                                                                   |       |                           |
| <b>Prerrequisitos:</b> 1. Cliente con plan activo de 12 sesiones en 4 semanas. 2. Sistema con lógica de validación de fechas. 3. Al menos una sesión cuya fecha de inicio de plan esté vencida.                                                                                       |       |                           |
| <b>Pasos:</b> 1. Autenticarse como cliente. 2. Acceder al módulo de Reservas. 3. Seleccionar una fecha de sesión dentro del período válido y confirmar. 4. Intentar reservar una sesión en una fecha posterior al vencimiento del plan. 5. Verificar el mensaje de error del sistema. |       |                           |
| <b>Resultado esperado:</b> La reserva dentro del período es aceptada. La reserva fuera del período es rechazada con mensaje informativo. Firestore no registra la sesión no válida.                                                                                                   |       |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                    |       |                           |

| Visualización de disponibilidad en tiempo real                                                                                                                                                                                                                                          | CP-29 | ¿Prueba de despliegue? No |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el cliente puede visualizar los horarios disponibles en tiempo real, reflejando los cupos actualizados automáticamente tras cada reserva.                                                                                                             |       |                           |
| <b>Prerrequisitos:</b> 1. Cliente autenticado. 2. Horarios configurados con cupos en Firestore. 3. Al menos un horario con cupos disponibles y otro sin cupos.                                                                                                                          |       |                           |
| <b>Pasos:</b> 1. Autenticarse como cliente. 2. Navegar a Ver Horarios / Disponibilidad. 3. Verificar que los horarios con cupo disponible se muestran habilitados. 4. Realizar una reserva y volver a la pantalla de disponibilidad. 5. Confirmar que el cupo disminuyó en tiempo real. |       |                           |
| <b>Resultado esperado:</b> Los horarios con cupos disponibles se muestran habilitados. Los horarios sin cupos se muestran como no disponibles. El contador de cupos se actualiza en tiempo real tras cada reserva.                                                                      |       |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                      |       |                           |

| Cancelación de reserva por cliente                                                                                                                                                                                                                                             | CP-30 | ¿Prueba de despliegue? No |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el cliente puede cancelar una reserva vigente dentro del período permitido y que el cupo es liberado automáticamente en el sistema.                                                                                                          |       |                           |
| <b>Prerrequisitos:</b> 1. Cliente autenticado con al menos una reserva activa. 2. Reserva dentro del período permitido de cancelación. 3. Reglas de cancelación configuradas.                                                                                                  |       |                           |
| <b>Pasos:</b> 1. Autenticarse como cliente. 2. Navegar a Mis Reservas. 3. Seleccionar una reserva vigente. 4. Presionar Cancelar Reserva. 5. Confirmar la acción. 6. Verificar que la reserva ya no aparece en el listado. 7. Verificar que el cupo fue liberado en la agenda. |       |                           |
| <b>Resultado esperado:</b> La reserva es eliminada correctamente de Firestore. El cupo del horario aumenta en 1. El cliente recibe confirmación de la cancelación.                                                                                                             |       |                           |

**Resultado obtenido:** Exitoso

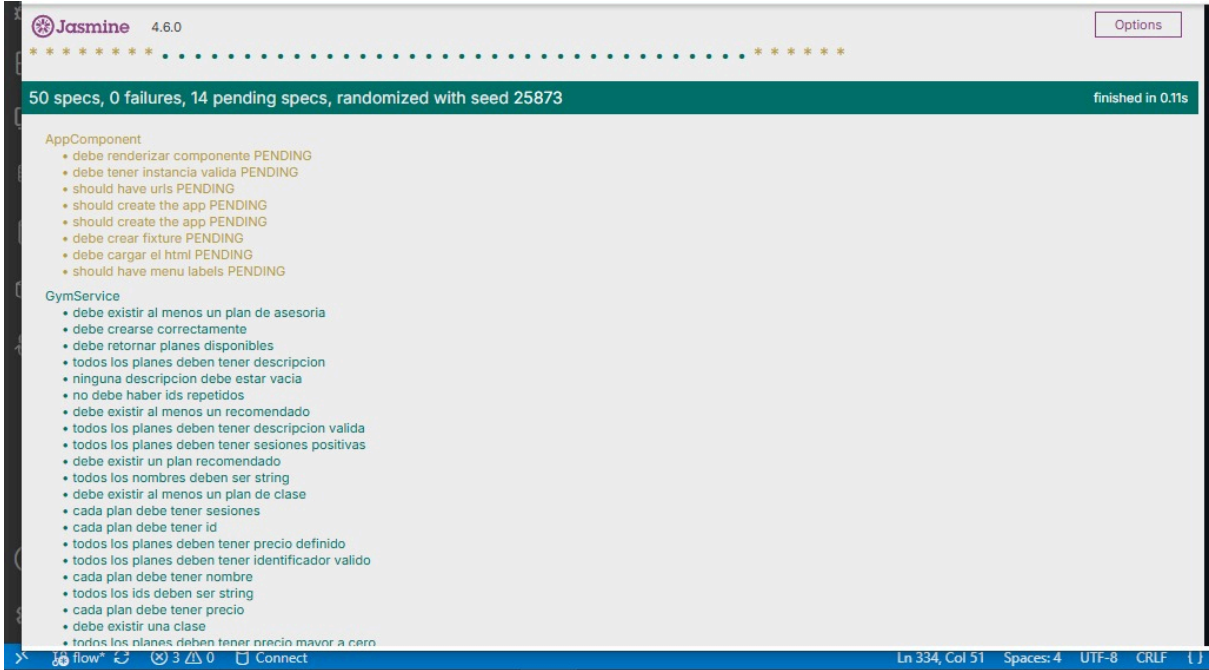
| Notificación de reserva confirmada y control de asistencia                                                                                                                                                                                                                                          | CP-31 | ¿Prueba de despliegue? No |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el sistema notifica al usuario al confirmar una reserva y que el administrador puede registrar la asistencia del cliente a la sesión agendada.                                                                                                                    |       |                           |
| <b>Prerrequisitos:</b> 1. Cliente autenticado con reserva activa confirmada. 2. Administrador autenticado. 3. Sistema de notificaciones habilitado.                                                                                                                                                 |       |                           |
| <b>Pasos:</b> 1. Realizar una reserva como cliente. 2. Verificar que el sistema muestra confirmación de reserva. 3. Verificar que se envía notificación al usuario. 4. Autenticarse como administrador. 5. Navegar al módulo de Asistencia. 6. Localizar la sesión y marcar asistencia del cliente. |       |                           |
| <b>Resultado esperado:</b> El cliente recibe notificación de confirmación de reserva. El administrador puede registrar asistencia. El registro queda guardado en Firestore.                                                                                                                         |       |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                                  |       |                           |

| Reportes, historial y gestión de reservas por administrador                                                                                                                                                                                                                                                                               | CP-32 | ¿Prueba de despliegue? No |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------|
| <b>Descripción:</b> Verificar que el administrador puede eliminar reservas en casos excepcionales, generar reportes de uso de sesiones y ocupación, y que el cliente puede consultar su historial de sesiones y pagos.                                                                                                                    |       |                           |
| <b>Prerrequisitos:</b> 1. Administrador autenticado. 2. Datos históricos de reservas y pagos disponibles en Firestore. 3. Módulo de reportes implementado.                                                                                                                                                                                |       |                           |
| <b>Pasos:</b> 1. Autenticarse como administrador. 2. Navegar a Gestión de Reservas y eliminar una reserva de prueba. 3. Acceder al módulo de Reportes. 4. Generar un reporte de ocupación por horario. 5. Autenticarse como cliente y acceder a Mi Historial. 6. Verificar que el historial de sesiones y pagos se muestra correctamente. |       |                           |
| <b>Resultado esperado:</b> La reserva eliminada desaparece del sistema. El reporte muestra datos actualizados de ocupación. El historial del cliente refleja sus sesiones y pagos correctamente.                                                                                                                                          |       |                           |
| <b>Resultado obtenido:</b> Exitoso                                                                                                                                                                                                                                                                                                        |       |                           |

# ESTRATEGIA DE EJECUCIÓN DE PRUEBAS

| Caso de Prueba | Ciclo 1 - Sprint 1 (Auth) | Ciclo 2 - Sprint 2 (Reservas) | Ciclo 3 - Sprint 3 (Pagos/Admin) | Ciclo 4 - Sprint 4 (Integración) |
|----------------|---------------------------|-------------------------------|----------------------------------|----------------------------------|
| CP-01          | X                         |                               |                                  |                                  |
| CP-02          | X                         |                               |                                  |                                  |
| CP-03          | X                         |                               |                                  |                                  |
| CP-04          |                           | X                             |                                  |                                  |
| CP-05          |                           | X                             |                                  |                                  |
| CP-06          |                           | X                             |                                  |                                  |
| CP-07          |                           |                               | X                                |                                  |
| CP-08          |                           | X                             |                                  | X                                |
| CP-09          |                           | X                             |                                  | X                                |
| CP-10          |                           |                               | X                                | X                                |
| CP-11          |                           |                               |                                  | X                                |
| CP-12          |                           |                               | X                                | X                                |

## Resultados de QA



El reporte automatizado detalla los resultados de las pruebas unitarias y de integración ejecutadas en el sistema GymOlympo utilizando el framework Jasmine (versión 4.6.0). Estas pruebas aseguran que los componentes de código funcionen exactamente bajo las reglas de negocio planificadas, previniendo la aparición de errores en producción.

## RESUMEN GENERAL DE LA EJECUCIÓN

- Total de especificaciones (pruebas): 50 especificaciones en total.
- Fallas detectadas: 0 fallas (cobertura exitosa en las pruebas activas).
- Pruebas pendientes: 14 pruebas en estado de espera o desarrollo (pending).
- Tiempo de ejecución: 0.11 segundos (rendimiento óptimo del entorno de pruebas).

## DETALLE DE PRUEBAS POR MÓDULO

**MÓDULO: APPCOMPONENT (INTERFAZ GRAFICA BASE)** Este bloque agrupa las pruebas encargadas de verificar que la aplicación Angular e Ionic cargue y construya la vista inicial correctamente. Actualmente, se encuentran en estado PENDIENTE, lo que significa que están escritas en el código pero deshabilitadas temporalmente mientras se estabiliza la lógica de datos:

- Validación de renderizado del componente principal.
- Verificación de instancia válida y creación del entorno (fixture).
- Comprobación de rutas (URLs) y carga del archivo HTML.
- Verificación de las etiquetas del menú global.

**MÓDULO: GYMSERVICE (LÓGICA DE NEGOCIO Y PLANES)** Este módulo superó con un 100% de éxito (color verde) todas las pruebas críticas asociadas al catálogo de membresías y ofertas comerciales del gimnasio, garantizando que no existan datos corruptos o huérfanos:

- Integridad del Modelo de Datos: Se comprobó con éxito que cada plan de entrenamiento contenga obligatoriamente un ID único, un nombre, un precio y un número de sesiones asignadas.
- Consistencia del Tipo de Datos: El sistema valida correctamente que todos los identificadores (IDs) y nombres de los planes sean cadenas de texto (String), evitando errores de lectura en la base de datos NoSQL.
- Validaciones Restrictivas de Negocio: Se verifica con éxito que ningún plan tenga un precio igual o menor a cero, que los cupos de sesiones sean siempre valores positivos y que no existan IDs duplicados o repetidos dentro del sistema.
- Control de Contenido Requerido: Las pruebas validan de forma estricta que todos los planes incluyan una descripción y que ninguna de estas descripciones esté vacía.
- Existencia de Oferta Mínima: El servicio responde correctamente asegurando que existan de manera mandatoria en la base de datos al menos un plan de asesoría, un plan de clase y un plan destacado como recomendado para el usuario.

## CONCLUSIÓN DEL INFORME DE QA

Los resultados demuestran que la capa lógica más sensible del proyecto GymOlympo (la gestión de planes, cobros y consistencia de datos de GymService) está completamente blindada contra fallas operativas básicas. Para las siguientes iteraciones del desarrollo, el equipo de QA debe enfocarse en activar y completar el set de pruebas del AppComponent para certificar que la experiencia visual del usuario final se despliegue sin inconvenientes en el navegador.

## RESUMEN GENERAL DE LA EJECUCIÓN

A lo largo del ciclo de desarrollo del sistema GymOlympo, el equipo ha identificado e implementado diversas mejoras técnicas, correcciones de errores críticos (bugs) y optimizaciones de diseño, las cuales han quedado registradas en el flujo de trabajo de QA y en la planificación del tablero Kanban de Trello.

A continuación se detallan los principales cambios aplicados para robustecer la estabilidad, seguridad y usabilidad de la plataforma:

**CORRECCIÓN DE ERRORES CRÍTICOS (BUGS):** Reparación del Bug de Mutación de Roles en el Login: Se detectó un error grave en la etapa de autenticación donde el sistema cambiaba automáticamente el rol del administrador a "cliente" cada vez que este iniciaba sesión en el frontend. Se corrigió la persistencia y la lógica de lectura en el servicio AuthService, asegurando que el token y los datos de la colección "users" en Cloud Firestore respeten de forma estricta los permisos correspondientes al perfil asignado.

#### MEJORAS DE USABILIDAD Y CONTROL DE CONCURRENCIA:

- **Bloqueo de Duplicación en la Creación de Horarios:** En el panel de administración, se implementó una optimización reactiva para evitar que los clics múltiples y consecutivos por parte del usuario generen registros duplicados de un mismo bloque de clase en la base de datos. Ahora el sistema bloquea el botón tras la primera pulsación mientras el HorarioService procesa la transacción en Firestore.
- **Integración de Identidad Visual (Branding):** Se integró de manera exitosa el logotipo oficial del gimnasio ("OLYMPO: Salud, Fuerza, Movimiento") en la interfaz del frontend utilizando los componentes de Ionic Angular, mejorando significativamente la estética y la experiencia de usuario dentro de las pantallas de acceso.
- **Desnormalización de Datos en Firestore (NoSQL):** Para contrarrestar la falta de uniones de tablas nativas (JOINS) por la naturaleza no relacional de Firebase, se aplicó la técnica de desnormalizar ciertos datos redundantes (como "clienteNombre", "fecha" y "hora") directamente dentro de la colección "reservas". Esto redujo de forma drástica la cantidad de consultas necesarias en el frontend, optimizando el rendimiento de la aplicación y disminuyendo los costos de lectura en la nube.

#### COBERTURA Y ASEGURAMIENTO DE CALIDAD (QA)

**Blindaje de Reglas de Negocio en la Oferta Comercial:** A través del framework Jasmine, se implementó una suite automatizada de 50 especificaciones. Se dio máxima prioridad al GymService, logrando validar con cero fallas que no existan planes sin precio, que no haya identificadores de membresía repetidos y que las descripciones nunca queden vacías, garantizando la consistencia total del catálogo comercial en la base de datos.

# Definición de metodología

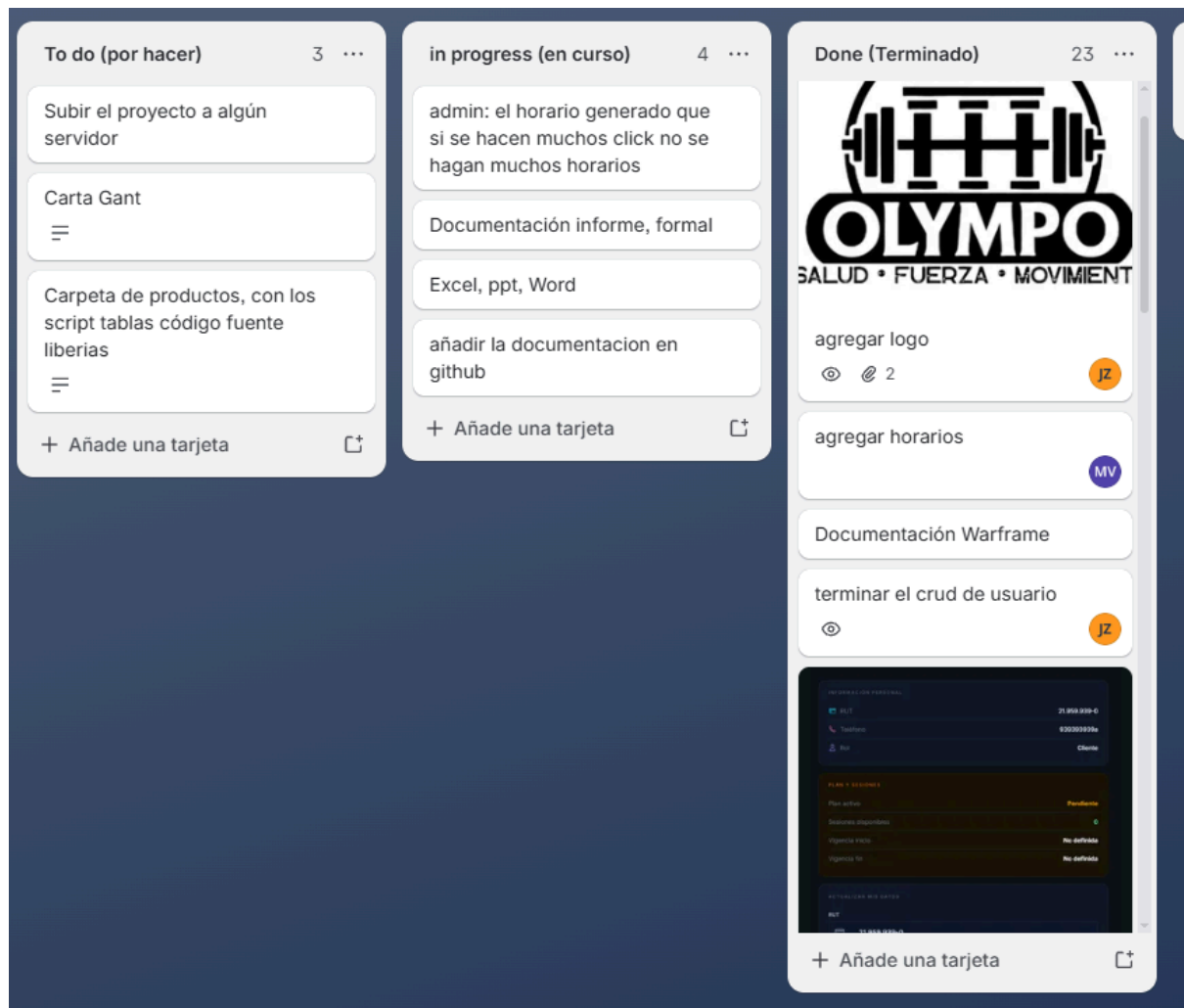
## Metodología de desarrollo (Scrum)

Se utilizará la metodología ágil Scrum para el desarrollo del proyecto, debido a su capacidad de adaptación a cambios y su enfoque en la entrega continua de valor durante todo el ciclo de desarrollo. El trabajo se organizará en sprints o iteraciones cortas, lo que permitirá planificar, ejecutar y evaluar el avance del sistema de forma progresiva, asegurando una mejora continua en el desarrollo de la plataforma.

Además, se establecerán reuniones formales con el cliente cada 15 días, con el objetivo de revisar avances, validar funcionalidades y realizar ajustes necesarios en base a nuevos requerimientos o cambios en las necesidades del sistema.

El cliente estará involucrado de manera constante durante el desarrollo del proyecto, permitiendo retroalimentación continua, incorporación de mejoras y ajustes funcionales, asegurando que la solución final se ajuste de manera precisa a sus expectativas y necesidades reales.

El tablero de Trello expuesto corresponde a la herramienta de gestión y planificación ágil (metodología Kanban) utilizada por el equipo de desarrollo para el proyecto GymOlympo. Permite coordinar las tareas, hacer el seguimiento del estado del software en tiempo real, asignar responsabilidades a los desarrolladores (identificados por sus iniciales como JZ y MV) y priorizar los requerimientos técnicos y correcciones de errores (bugs).



## Roles del Proyecto

| Rol                                       | Responsabilidad                                                                               | Integrante       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------|------------------|
| Product Owner                             | Definir requerimientos y validar funcionalidades con el cliente                               | Manuel           |
| Scrum Master,<br>Desarrollador Full Stack | Coordinar actividades y asegurar el cumplimiento de Scrum                                     | Abraham Gonzalez |
| Desarrollador Full Stack, dba, analista   | Desarrollo frontend, backend y base de datos, documentación, análisis de requerimiento        | Joscelynne Díaz  |
| Desarrollador Frontend y testing          | Apoyo en desarrollo e integración de funcionalidades y testeo de la aplicación, documentación | Marcelo Mancilla |

Visualización final del rendimiento

## Patrón arquitectónico basado en Componentes y Servicios

La arquitectura de GymOlympo se basa en un enfoque orientado a componentes y servicios, permitiendo una adecuada separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y el acceso a los datos.

En el frontend, desarrollado con Angular e Ionic, la aplicación se organiza mediante componentes reutilizables encargados de la presentación y la interacción con el usuario. Cada componente posee una responsabilidad específica, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

La lógica de comunicación con el backend se implementa mediante servicios de Angular, los cuales gestionan las solicitudes HTTP, el manejo de la autenticación y el intercambio de información con los servicios de Firebase. Esta separación permite mantener los componentes enfocados en la interfaz de usuario mientras los servicios centralizan la lógica de acceso a datos.

Esta arquitectura basada en componentes y servicios favorece la modularidad, la mantenibilidad y la escalabilidad del sistema, permitiendo incorporar nuevas funcionalidades de manera ordenada y facilitando la evolución futura de la plataforma GymOlympo.

## Documentación y entregable

Durante el desarrollo del proyecto se generará la siguiente documentación:

- Documentos de requerimientos.
- Diseño de arquitectura.
- Documentación de APIs.
- Plan de prueba.
- Entrega final.

Esta documentación permite asegurar la trazabilidad del proyecto, facilitar su mantenimiento y garantizar el cumplimiento de estándares de calidad.

Además, se incluirán planes de prueba, los cuales permiten validar el correcto funcionamiento del sistema mediante pruebas funcionales y de integración. Estas pruebas se realizarán de manera iterativa en cada sprint, asegurando la calidad del software durante su desarrollo.

Como parte de la gestión del proyecto, se registrarán entregas parciales o incrementales, correspondientes a los avances funcionales del sistema en cada iteración, permitiendo evidenciar el progreso del producto de software y realizar ajustes según retroalimentación del cliente.

Finalmente, la entrega final contempla el sistema completamente funcional, junto con su documentación técnica completa, asegurando su correcto despliegue, funcionamiento y mantenimiento.

# Evidencia de Avance del Proyecto

Con el objetivo de mantener trazabilidad y control sobre el desarrollo del sistema, se utilizó GitHub como herramienta principal de control de versiones y trabajo colaborativo.

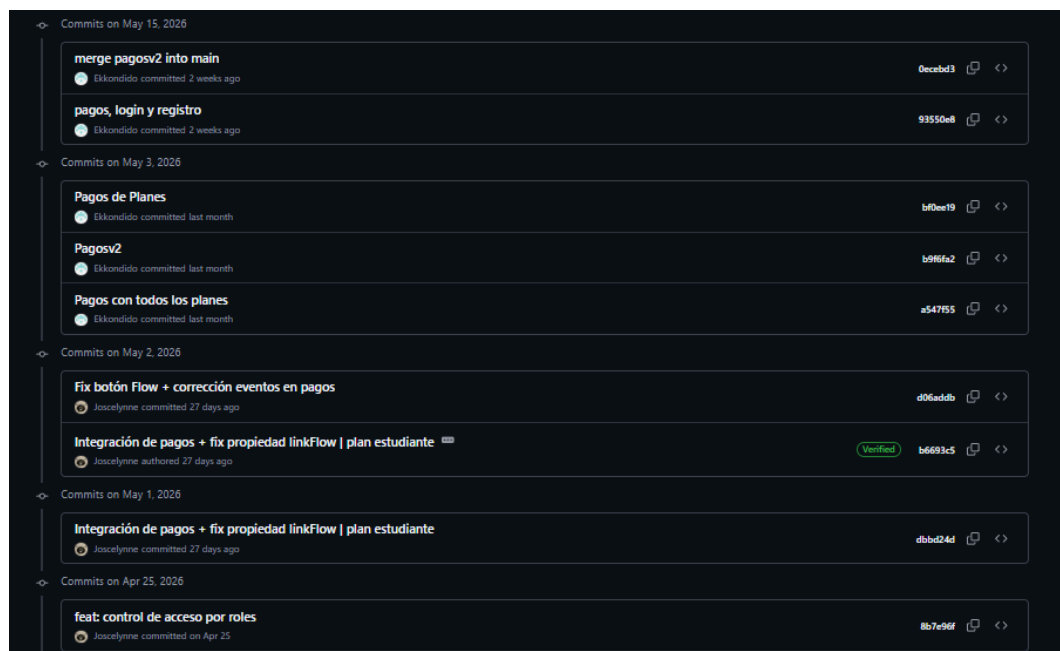
Durante el desarrollo del proyecto se implementó una metodología de trabajo basada en ramas y commits progresivos, permitiendo desarrollar funcionalidades de manera incremental y mantener un registro detallado de los cambios realizados dentro del sistema.

Entre los avances implementados actualmente se encuentran:

- Integración entre frontend y backend.
- Implementación de autenticación de usuarios.
- Desarrollo de módulos de reservas y gestión de planes.
- Configuración inicial de infraestructura cloud.
- Implementación de control de acceso y validación de usuarios.

El uso de GitHub permitió organizar el flujo de trabajo del equipo, facilitar la colaboración entre integrantes y mantener seguimiento continuo sobre el estado de avance del proyecto.

Asimismo, el registro de commits y actualizaciones permitió evidenciar el desarrollo progresivo de funcionalidades dentro de cada sprint definido mediante metodología Scrum.



# Documentación de Pruebas Realizadas

Durante el desarrollo del proyecto se ejecutaron distintas pruebas funcionales y de integración con el objetivo de validar el correcto funcionamiento de la plataforma Gym Olympo.

Las pruebas realizadas permitieron verificar la comunicación entre frontend, backend y servicios cloud, además de evaluar el comportamiento de funcionalidades críticas del sistema.

| Prueba realizada                   | Resultado |
|------------------------------------|-----------|
| Inicio de sesión de usuarios       | Exitosa   |
| Registro de usuarios               | Exitosa   |
| Persistencia de datos en Firestore | Exitosa   |
| Comunicación frontend/backend      | Exitosa   |
| Validación de autenticación        | Exitosa   |
| Gestión de reservas                | Exitosa   |
| Integración de servicios cloud     | Exitosa   |
| Validación de roles de usuario     | Exitosa   |

Los resultados obtenidos permitieron detectar errores de integración durante etapas tempranas del desarrollo, facilitando la corrección oportuna de incidencias y mejorando la estabilidad general de la solución.

Las pruebas fueron ejecutadas de manera iterativa durante los distintos sprints del proyecto, permitiendo validar continuamente las funcionalidades desarrolladas.

# Stakeholders Identificados

Manuel (Dueño de GymOlympo): Responsable de definir las necesidades del negocio, validar los requerimientos y aprobar la solución desarrollada.

Equipo de Desarrollo: Responsables del análisis, diseño, implementación, pruebas y mantenimiento de la solución.

Docente Evaluador (Cristian): Encargado de supervisar el cumplimiento de los objetivos académicos y evaluar el desarrollo del proyecto.

## Conclusión

El desarrollo de GymOlympo permitió abordar una problemática real relacionada con la gestión manual de reservas, sesiones y horarios dentro de un gimnasio, proponiendo una solución tecnológica orientada a optimizar los procesos internos y mejorar la experiencia de los usuarios. A través de la implementación de una plataforma digital, se busca reducir la carga administrativa, minimizar errores operativos y proporcionar un servicio más eficiente, accesible y organizado.

Desde el punto de vista técnico, la solución se fundamenta en una arquitectura serverless basada en Firebase y en un enfoque arquitectónico orientado a componentes y servicios. Esta estructura permite una adecuada separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y la persistencia de datos, favoreciendo la mantenibilidad, escalabilidad y seguridad del sistema. Asimismo, el uso de tecnologías como Angular, Ionic, Firebase Authentication y Cloud Firestore garantiza una infraestructura moderna, flexible y alineada con los requerimientos actuales del proyecto.

La aplicación de la metodología Scrum permitió organizar el trabajo de manera iterativa e incremental, facilitando la planificación de tareas, el seguimiento del avance y la incorporación continua de mejoras durante el desarrollo. Del mismo modo, el uso de herramientas de control de versiones y gestión colaborativa contribuyó a mantener la trazabilidad y calidad del proyecto.

Finalmente, las pruebas realizadas permitieron validar la integración de los distintos componentes del sistema y comprobar el correcto funcionamiento de las funcionalidades implementadas. En conjunto, los resultados obtenidos demuestran la factibilidad técnica de la solución propuesta y establecen una base sólida para futuras ampliaciones, integraciones y mejoras que permitan continuar fortaleciendo los servicios ofrecidos por GymOlympo.

## Bibliografía

Google. (s.f.). Firebase Documentation. Recuperado de <https://firebase.google.com/docs>

Google. (s.f.). Cloud Firestore Documentation. Recuperado de <https://firebase.google.com/docs/firestore>

Google. (s.f.). Firebase Authentication Documentation. Recuperado de <https://firebase.google.com/docs/auth>

Angular Team. (s.f.). Angular Documentation. Recuperado de <https://angular.io/docs>

Ionic Team. (s.f.). Ionic Framework Documentation. Recuperado de <https://ionicframework.com/docs>

Capacitor Team. (s.f.). Capacitor Documentation. Recuperado de <https://capacitorjs.com/docs>

Microsoft. (s.f.). TypeScript Documentation. Recuperado de <https://www.typescriptlang.org/docs>

GitHub, Inc. (s.f.). GitHub Documentation. Recuperado de <https://docs.github.com>

Atlassian. (s.f.). Trello Guide. Recuperado de <https://support.atlassian.com/trello>

Postman, Inc. (s.f.). Postman Learning Center. Recuperado de <https://learning.postman.com>

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Recuperado de <https://scrumguides.org>

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9.<sup>a</sup> ed.). McGraw-Hill.

Sommerville, I. (2016). Ingeniería de software (10.<sup>a</sup> ed.). Pearson Educación.